

# Project Final Report

## Travelog

### Team 4

Carolyna Sias ([carolyna.sias@sjsu.edu](mailto:carolyna.sias@sjsu.edu)), Meghana Indukuri ([meghana.indukuri@sjsu.edu](mailto:meghana.indukuri@sjsu.edu)), Isaiah Villanueva ([isaiah.villanueva@sjsu.edu](mailto:isaiah.villanueva@sjsu.edu))

## Table of Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| Functional Requirements -----                               | 3  |
| Entities, Attributes, relationships with Multiplicity ----- | 8  |
| Entity-Relationship Diagram -----                           | 14 |
| Relational Schemas -----                                    | 15 |
| MySQL Workbench Tables -----                                | 16 |
| Project Implementation-----                                 | 32 |
| Lessons Learned -----                                       | 67 |

## Functional Requirements

For this application, we introduce three user views: Traveler, Company, and Administrator.

### Traveler View:

**Critically, on Travelog, a traveler is able to register, login, logout, and delete their account.**

#### **Register Account**

Description:

A traveler can create a new account on Travelog. The system will collect registration information, validate it, and store a new record in the Users table on the database. Duplicate accounts will not be allowed via unique constraints on the username and the email account. For security purposes, the password will also be encrypted before being inputted into the database.

Input:

The registration form will have the following fields:

**Username** (String, Required, Unique)

**Email Address** (String, Required, Unique, Valid Email Format)

**Password** (String, Required, Encrypted Before Storage)

Process:

1. The traveler accesses the registration page and will enter the information into the form fields.
2. The system will validate that all required fields are complete
3. The system verifies that username and email address does not already exist in the Users table, the email format is valid, and that the password meets required security measures (e.g., length and alphanumeric constraints).
4. If validation passes, then the system will encrypt the password, and insert a new record into the users table containing a username, email, and encrypted password. A confirmation message will also be displayed to the user and they will be redirected to the homepage of the application.
5. If the validation fails, then the system will cancel the registration process and show the corresponding error message.

#### **Account Sign In**

Description:

A traveler can login to an existing account on Travelog. The system will authenticate the user by comparing the collected login information with the records in the Users table on the database. If the authentication is successful, then the user will be able to access the homepage via a secure session.

Input:

The login form will have the following fields:

**Username** (String, Required)

**Password** (String, Required)

Process:

1. The traveler will enter their username and password into the login fields.
2. The system will validate that all required fields are complete
3. The system will pull the matching record from the User table in the database.
4. If the corresponding record is found, the system will redirect the user to the homepage via a secure session.
5. If the corresponding record is not found, the system will display the error message and the login is cancelled.

#### **Account Logout**

Description:

A traveller can logout of their account and their secure session will be ended by the system.

Input:

**Logout** (Button)

Process:

1. The traveler will click the logout button.
2. The system will terminate the current session.
3. The traveler is redirected to the login page.

### Delete Account

Description:

A traveller can delete their account and remove all their data and information from Travelog.

Input:

**Password** (String, Required)

Process:

1. Traveller will click the delete account button
2. The system will give a confirmation message where the user needs to input their password
3. The system will compare the password with records in the User table
4. If the corresponding match is found, the system will delete the record from the User table, and perform a cascading delete to remove any associated records.
5. If the match is not found (i.e., the password is incorrect), then the system will cancel deletion and show an error message.

**On Travelog, a traveler is also able to view and create trips, and pick corresponding destinations and dates for each one.**

### View trips

Description:

A traveller can view all the trips they have created thus far on a personal dashboard. They can see their planned, booked and cancelled trips all in one place.

Input:

**Secure User Session** (No Manual Input)

Process:

1. The system will do an SQL query for all the trips under the current user's id in the User\_Trips table.
2. If the query is empty, a message stating no trips created will be displayed.
3. If the query has results, the trips will be displayed on the dashboard for the user in descending order by status first (i.e., planned, booked, and cancelled), then by date.

### Create trip

Description:

A traveller can create a new trip that will later contain all the trip details (i.e., destination, dates, accommodations, etc.)

Input:

**Trip Name** (String, Required)

Process:

1. Traveler will click create trip
2. Traveler will enter the trip's name
3. The system will then enter the trip as a new record in the User\_Trips table.
4. The traveler will be redirected into the trip's homepage.

### Select Starting and Final destinations

Description:

A traveller can select their starting origin and their destination for the trip they created.

Input:

**Trip ID** (No Manual Input)

**Starting Location** (String, Required, Pick From List or Search)

**Final Destination** (String, Required, Pick From List or Search)

Process:

1. Travelers will search or pick destinations. For a search, the system will do an internal query to match the search.
2. The traveler will select and confirm the origin and destination.



3. The system updates the corresponding user\_trips row in the database and the changes show on the trip's homepage.

### Select preferred dates

Description:

A traveller can select their preferred dates for the trip they created.

Input:

**Trip ID** (No Manual Input)  
**Start Date** (Date, Required)  
**End Date** (Date, Required)

Process:

1. The traveler picks the start and end dates using a date picker
2. The system validates that the end date is not before the start date and is after the current date.
3. The system updates the corresponding user\_trips row in the database and the changes show on the trip's homepage.

**On Travelog, a traveler is also able to browse and filter transport options, confirm their choices, and update transport plans to account for changes.**

### Browse and filter transport options

Description:

A traveller can view the different flight options that work with the trip's date range and destination. Flight data is pulled from Google Flight results using an API. Results can also be filtered based on certain criteria.

Input:

**Trip ID** (No Manual Input)  
**Selected Filters: Cost, Time** (Optional)

Process:

1. The traveller searches in the search bar or selects filters for outbound and inbound transportation.
2. The system will do a query for transport options based on the travellers search, selected filters, destination, and dates.
3. The system will display all corresponding and filtered results.

### Select transportation

Description:

A traveller can select their transportation for both trips on the journey (to and from).

Input:

**Trip ID** (No Manual Input)  
**Selected Transport** (Transport ID (automatic based on selection), Required)  
**Direction** (Required, Return or Outbound)

Process:

1. The traveller selects the transportation option they like and clicks confirm.
2. The system fills out a record with all the information and adds it to the trip\_transport table.
3. If a transportation choice is already in the table, it will overwrite the previous record. This works as an update option for travellers.

**On Travelog, a traveler is also able to browse and filter hotel options, confirm their choices, and update hotel plans to account for changes.**

### Browse and filter hotel options

The same logic as "Browse and filter transport options" applies here. Refer to the explanation for that functional requirements and replace transportation with hotel. Filters include number of adult guests, rating, and price per night. Listings from Google Hotels API will be displayed alongside Company special offers postings.

### Select hotel

Description:

A traveller can select their hotel for their trip.

Input:

**Trip ID** (No Manual Input)

**Selected Hotel** (Hotel ID (automatic based on selection), Required)

Process:

1. The traveler selects a hotel and enters the specific dates.
2. The system validates the dates are feasible.
3. The system adds a record into the trip\_hotels with the required information.

**On Travelog, a traveler is able to confirm their booking and move their trip from a planned to booked status. They can also then cancel their booking, and move their booked status to cancelled. They may possibly receive a refund as well.**

### **Review and Confirm Trip Booking**

Description:

The traveler can view their cost summary of the trip (both hotel and transportation) and then finalize the booking.

Input:

**Trip ID** (No Manual Input)

**Payment Method** (Form, Required)

Process:

1. The system queries the Trip ID to get the correct information from the trip\_transport and trip\_hotels tables.
2. The costs pulled from the transportation and hotel tables will be summarized and calculated by the system.
3. The total summary will be shown to the traveller who can either continue with the booking or cancel and return to the trip homepage.
4. If they continue, the traveller selects their payment method and fills it out, here security is critical.
5. A new record will be created in the Payments table which will not contain any sensitive information but rather metadata about the traveller's payment for the booking.
6. The trip status is then updated from planned to booked.

### **Cancel Trip and Receive Refund**

Description:

The traveller can cancel the entire trip and receive a full refund.

Input:

**Trip ID** (No Manual Input)

**Password** (String, Required)

Process:

1. The traveler selects the level of which they are cancelling.
2. The system allows them to make a more detailed selection of what they are cancelling (i.e., one of the booked hotels or transportation).
3. The system calculates expected refunds, the traveller can then continue or cancel their cancellation.
4. The traveller enters their password to confirm their decision if they do cancel.
5. The system updates the status accordingly in all corresponding records (trips, trip\_hotels, trip\_transportations, payments).

Company View:

**Critically, on Travelog companies can register, login, logout, and delete their account.**

### **Register Account, Account Sign In, Account Signout, Delete Account**

Fundamentally, there is no difference between this and the previously listed requirements for Travellers. Thus, to understand the register account, login, logout, and delete functionalities for companies refer to the traveler view above. One major change is that for an account created under a company, it will be manually audited by the Travelog personnel.

**Furthermore, on Travelog, companies can post unseen offers, availability, and discounts for Travelog travelers. They can also delete any old or outdated offers.**

### **Post Offer**

Description:

Companies can post or update existing posts under their brand for Travelog users. These updates and offers could include things like discounts and new availability.

Input:

Destination (String, Required)  
 Name (String, Required)  
 Cost (String, Required)  
 Availabilities (String, Required)  
 Discounts (String, Required)

Process:

1. Company clicks on create or manage postings on the homepage.
2. Company enters modified information or new information for a posting.
3. System verifies all information is valid per constraints and expectations.
4. New postings will be added to the hotels or transportation tables by the system, updates will update existing records.
5. Confirmation is shown to the company once the posting is successfully added.

### **Delete Post**

Description:

Companies can delete existing posts from Travelog so that users can no longer see it.

Input:

Posting ID (No Manual Input, Required)  
 Password (String, Required)

Process:

1. Company clicks on an existing post to delete.
2. Company clicks on delete confirmation.
3. The system prompts the company to enter a password. Validation by system occurs, failure cancels the deletion process.
4. If the company is successful in confirming deletion, the posting is removed from Travelog users except those who have already booked the accommodations or transportation.
5. The system archives records using a status column rather than a delete to avoid voiding existing users who have previously booked.

### Administrator View:

**Admins will have existing accounts, which they will simply use to login. Login functionality is equivalent to any regular user's experience logging in. Thus, the only important functionality these users have is confirming company accounts and handling user queries.**

### **Confirm or Reject Company Accounts**

Description:

Administrators can review to approve or deny a company account on the platform.

Input:

Selected Company (Click on Company)  
 Approval Choice (Accept or Deny, Required)

Process:

1. Administrator sees list of accounts awaiting approval and selects company for review.
2. The administrator chooses to deny or accept.
3. The status of the company account is changed by the system accordingly, and an email notification to the company is sent out regarding the decision.

4. The list of accounts is now reduced by one through system update of the list.

### Handle User Queries

Description:

Administrators can deal with user accounts and issues sent through queries on the application.

Input:

Selected query (Required)

Administrative Action (Required)

Process:

1. Administrator selects a query in their inbox
2. The system retrieves the query from the Query table
3. The Administrator makes an action based on the query
4. The system records any changes made including updates to the database.

## Entities, Attributes, Relationships with Multiplicity

For this application, we introduce 14 major entities:

### Users

This entity is for storing all the information about user accounts in the application.

The primary key will be: User\_ID

Attributes will be as follows:

- (1) User\_ID
- (2) Username
- (3) Email
- (4) Password\_Hash
- (5) TimeStamp\_Created
- (6) Account\_Status

Each attribute plays an important role, username will be placed under a unique constraint as well as email. The password will be stored as a hash per common privacy protocols. A timestamp for creation will also be stored for both data collection and possible benefits for long-term users. Account status will help determine if the account is active, suspended, pending (like when the user needs to confirm password via email) or deactivated.

### Relationships with multiplicity:

- (1) A user can only belong to one subclass entity: Traveler, Company, or Administrator
- (2) A user can submit zero or many support queries (Support\_Queries)
- (3) A user can make zero or many payments (Payments)
- (4) A user can own zero or many trips (Trips) through the Traveler subclass.

### Travelers

This entity is for storing traveler-specific information. This is a subclass of the Users entity.

The primary key will be: User\_ID

Attributes will be as follows:

- (1) User\_ID
- (2) First\_Name
- (3) Last\_Name
- (4) Default\_Departure\_Location

Each attribute plays an important role, User\_ID is the primary key and the foreign key for this entity set. First and last name are for some UI related information/personalization. Default\_Departure\_Location helps users spend less time picking a starting location for their trips.

**Relationships with multiplicity:**

- (1) A traveler has to be exactly one user (User)
- (2) A traveler can create zero or many trips (Trips)
- (3) A traveler can make zero or many payments (Payments)
- (4) A traveler can submit zero or many support queries (Support\_Queries)

**Companies**

This entity is for storing company-specific information. This is a subclass of the Users entity.

The primary key will be: User\_ID

Attributes will be as follows:

- (1) User\_ID
- (2) Company\_Name
- (3) Buisness\_Type
- (4) Approval\_Status
- (5) Contact\_Phone
- (6) Verification\_Date

User\_ID is both the foreign key and primary key for this entity set. The company\_name is important to help relay information about the company's public name. The business type will help inform about whether they provide transport, hotels, or even both. The approval status is for the review process in which the companies must be accepted onto the platform by an admin. Contact phone can help for the review process, and verification date will keep metadata about the review process as well.

**Relationships with multiplicity:**

- (1) A company has to be exactly one user (User)
- (2) A company can own zero or many hotel listings (Hotel\_Listings)
- (3) A company can own zero or many transportation listings (Transportation\_Listings)
- (4) A company can receive zero or many company approval reviews (Company\_Approval\_Reviews)
- (5) A company can submit zero or many support queries (Support\_Queries)

**Administrators**

This entity is for storing administrator-specific information. This is a subclass of the Users entity.

The primary key will be: User\_ID

Attributes will be as follows:

- (1) User\_ID
- (2) Admin\_Level

User\_ID is both the foreign key and primary key for this entity set. The admin\_level determines the administrative power this specific administrator holds.

**Relationships with multiplicity:**

- (1) An administrator has to be exactly one user (User)
- (2) An administrator can handle zero or many support queries (Support\_Queries)
- (3) An administrator can create zero or many company approval reviews (Company\_Approval\_Reviews)

**Trips**

This entity is for storing all the information about trips, or in other words the user's travel plans. This is created by users.

The primary key will be: Trip\_ID

Attributes will be as follows:

- (1) Trip\_ID
- (2) User\_ID

- (3) Trip\_Name
- (4) Start\_Location
- (5) End\_Location
- (6) Start\_Date
- (7) End\_Date
- (8) Status
- (9) Timestamp\_Created

Here Trip\_ID is the primary key, but User\_ID acts as a foreign key (as each trip belongs to a specific user). The trip\_name can be any value and is not constrained by uniqueness. The start location and end location will be critical for the trip but also do not have a unique constraint. The same applies to the start and end date. Status helps determine if the trip is planned, booked, or cancelled. The timestamp created provides extra metadata about the trip.

**Relationships with multiplicity:**

- (1) A trip must belong to exactly one traveler (Traveler)
- (2) A trip can have zero or many trip hotels (Trip\_Hotels)
- (3) A trip can have zero or many trip transportations (Trip\_Transportations)
- (4) A trip can have zero or many payments (Payments)
- (5) A trip can have zero or many refunds (Refunds) via payments

### **Hotel\_Listings**

This entity is for storing all the hotel listings posted by approved company accounts or scraped from trusted providers.

The primary key will be: Hotel\_ID

Attributes will be as follows:

- (1) Hotel\_ID
- (2) Company\_User\_ID (optional)
- (3) Hotel\_Name
- (4) Location
- (5) Description
- (6) Rating
- (7) Price\_Per\_Night
- (8) Availability\_Status
- (9) Listing\_Status

Hotel\_ID is the primary key here used to identify each listing uniquely. Company\_User\_ID is a foreign key referencing the Company entity, but it is optional (as some of the listings may be scraped from trusted sources).

Hotel\_Name, Location and Description are metadata for the users, alongside rating. Price per night is the price for travelers, availability status shows if there are rooms available to be booked. Listing status is for archived listings or hidden listings (easier to manage rather than deletion in case one user has them booked but another shouldn't be able to do so).

**Relationships with multiplicity:**

- (1) A hotel listing can belong to zero or one company (Companies)
- (2) A hotel listing can appear in zero or many trip hotels (Trip\_Hotels)

### **Transportation\_Listings**

This entity is for storing all the transportation listings posted by approved company accounts or scraped from trusted providers.

The primary key will be: Transportation\_ID

Attributes will be as follows:

- (1) Transportation\_ID

- (2) Company\_User\_ID (optional)
- (3) Transport\_Type
- (4) Transport\_Name
- (5) Departure\_Location
- (6) Arrival\_Location
- (7) Departure\_Time
- (8) Arrival\_Time
- (9) Base\_Price
- (10) Availability\_Status
- (11) Listing\_Status

Transportation\_ID is the primary key here used to identify each listing uniquely. Company\_User\_ID is a foreign key referencing the Company entity, but it is optional (as some of the listings may be scraped from trusted sources).

Transport type, name, departure location, arrival location, times, and base price are metadata for the users to help make a choice. Availability status shows if there are seats for booking available. Listing status is for archived listings or hidden listings (easier to manage rather than deletion in case one user has them booked but another shouldn't be able to do so).

**Relationships with multiplicity:**

- (1) A transportation listing can belong to zero or one company (Companies)
- (2) A transportation listing can appear in zero or many trip transportations (Trip\_Transportation)

## **Trip\_Hotels**

This entity is for storing all the hotels for a specific trip. This is a weak entity set because it depends on the Trip and Hotel\_Listing entities.

The primary key will be: Trip\_Hotel\_Sequence\_Num, Trip\_ID, Hotel\_ID

Attributes will be as follows:

- (1) Trip\_Hotel\_Sequence\_Num
- (2) Trip\_ID
- (3) Hotel\_ID
- (4) Check\_In\_Date
- (5) Check\_Out\_Date
- (6) Booked\_Price\_Per\_Night
- (7) Total\_Cost
- (8) Booking\_Status

The Trip\_Hotel\_Sequence\_Num is a part of the primary key used to list each hotel for the trip (each trip will have a sequence of hotels that are being used). The Trip\_Id is the foreign key referencing the Trip entity. Hotel\_ID is also a foreign key that references the Hotel\_Listings entity. The combination of these three keys forms the primary key for this entity. Check in date & checkout date are important metadata information for users who have booked or plan on booking the hotel. We also have booked prices per night as prices change over time, and what was booked prior may have had a cheaper price per night. The total cost shows how much will be spent, we store it as its own column because it includes all extra fees. Booking status helps determine if the booking is planned, booked, cancelled or even refunded.

**Relationships with multiplicity:**

- (1) A trip\_hotel has to belong to exactly one trip (Trips)
- (2) A trip\_hotel has to have exactly one hotel listing (Hotel\_Listings)
- (3) A trip on the other hand can have zero or many trip hotels (Trip\_Hotels)
- (4) A hotel listing on the other hand can have zero or many trip hotels (Trip\_Hotels)

## **Trip\_Transportation**

This entity is for storing all the transportation for a specific trip. This is a weak entity set because it depends on the Trip and Transport\_Listing entities.

The primary key will be: Trip\_Transport\_Sequence\_Num, Trip\_ID, Hotel\_ID

Attributes will be as follows:

- (1) Trip\_Transport\_Sequence\_Num
- (2) Trip\_ID
- (3) Transport\_ID
- (4) Direction
- (5) Price
- (6) Booking\_Status

The Trip\_Transport\_Sequence\_Num is a part of the primary key used to list each transport for the trip (each trip will have a sequence of transportations that are being used). The Trip\_ID is the foreign key referencing the Trip entity. Transport\_ID is also a foreign key that references the Transport\_Listings entity. The combination of these three keys forms the primary key for this entity. The direction provides important information about which way (outbound or inbound) for the travelers. The price once again may change so we store it as an attribute, and the booking status helps us determine if the booking is planned, booked, cancelled or even refunded.

**Relationships with multiplicity:**

- (1) A trip\_transport has to belong to exactly one trip (Trips)
- (2) A trip\_transport has to have exactly one transport listing (Transport\_Listings)
- (3) A trip on the other hand can have zero or many trip transports (Trip\_Transports)
- (4) A transport listing on the other hand can have zero or many trip transports (Trip\_Transports)

## Payments

This entity is for storing all the information regarding payments for a specific booked trip transport or hotel.

The primary key will be: Payment\_ID

The attributes are as follows:

- (1) Payment\_ID
- (2) Trip\_ID
- (3) User\_ID
- (4) Amount
- (5) Payment\_Method
- (6) Payment\_Status
- (7) Payment\_Timestamp

The Payment\_ID is the primary key for this entity list. The Trip\_ID is a foreign key referencing the Trip entity. The User\_ID is a foreign key referencing the User entity table. The amount is the amount that the user pays, and the payment method highlights which type of payment they did (VISA, American Express, and beyond). The payment status helps determine the status of the payment (i.e., refunded or paid or declined). The timestamp shows when the payment occurred and helps with good metadata for payment transactions.

**Relationships with multiplicity:**

- (1) A payment must belong to exactly one trip (Trips)
- (2) A payment must be made by exactly one user (Users)
- (3) A trip can have zero or many payments (Payments)
- (4) A user can have zero or many payments (Payments)
- (5) A payment can have zero or many refunds (Refunds)

## Refunds

This entity is for storing all the information regarding refunds for a specific payment done for a specific booking.

The primary key will be: Refund\_ID

The attributes are as follows:



- (1) Refund\_ID
- (2) Payment\_ID
- (3) Refund\_Amount
- (4) Refund\_Reason
- (5) Refund\_Timestamp

The refund\_id is the primary key for this entity, while the payment\_id is the foreign key for this entity. The refund amount shows how much of the original payment is paid back to the user, and the refund reason helps explain why for logging reasons. The timestamp is also good additional metadata for admins and the company Travelog in general.

**Relationships with multiplicity:**

- (1) A refund belongs to exactly one payment (Payments)
- (2) A payment can have zero or many refunds (Refunds)

### Support\_Queries

This entity is for storing all the information regarding refunds for a specific payment done for a specific booking.

The primary key will be: Query\_ID

The attributes are as follows:

- (1) Query\_ID
- (2) User\_ID
- (3) Assigned\_Admin\_User\_ID
- (4) Subject
- (5) Message\_Body
- (6) Query\_Status
- (7) Created\_TimeStamp
- (8) Resolved\_TimeStamp

Here, Query\_ID is a primary key for this entity. The user\_id is the foreign key which is pulled from the User table. This is because the query must be from some user account. Assigned\_Admin\_User\_ID is the admin who is assigned to the query (which is optional) but is pulled from the Administrators table. Subject and message body highlight the information about the query. The status shows whether the query is completed, in progress, or done. The timestamps are additional metadata that we can use for the query information.

**Relationships with multiplicity:**

- (1) A support query must be submitted by exactly one user (User)
- (2) A support query can be assigned to 0 or 1 administrator (Administrators)
- (3) A user can submit zero or many support queries (Support\_Queries)
- (4) An administrator can handle zero or many support queries (Support\_Queries)

### Company\_Approval\_Reviews

This entity is for storing all the review actions by administrators for companies.

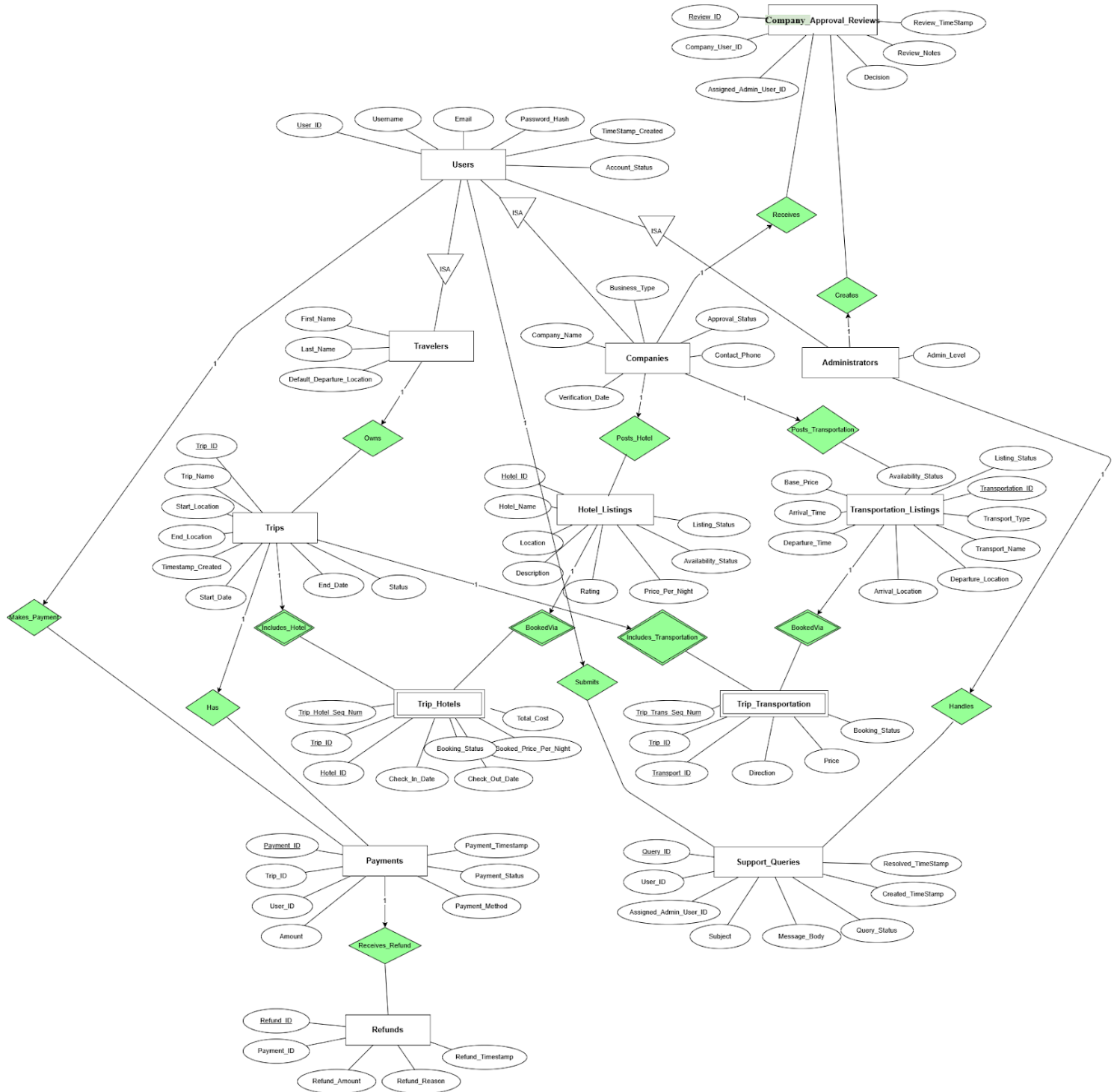
The primary key will be: Review\_ID

The attributes are as follows:

- (1) Review\_ID
- (2) Company\_User\_ID
- (3) Assigned\_Admin\_User\_ID
- (4) Decision
- (5) Review\_Notes
- (6) Review\_TimeStamp

# Entity-Relationship Diagram

Link to Full Diagram: [link](#)



## Relational Schemas

### Entities:

**User**(userID, email, username, password, account\_created, account\_status)

**Traveler**(userID, FirstName, LastName, current\_location)

**Company**(userID, name, business\_type, approval\_status, phone, verification\_date)

**Admin**(userID, admin\_level)

**Trip**(tripID, userID, trip\_name, start\_location, destination, start\_date, end\_date, status, time\_created)

**Hotel\_Listing**(hotelID, company\_userID, hotel\_name, location, description, rating, price\_per\_night, availability, listing\_status)

**Transport\_Listing**(transportID, company\_userID, transport\_type, transport\_name, departure\_location, arrival\_destination, departure\_time, arrival\_time, base\_cost, availability, listing\_status)

**Trip\_Hotels**(trip\_hotel\_sequence\_num, tripID, hotelID, check\_in\_date, check\_out\_date, booked\_price\_per\_night, total\_cost, booking\_status)

**Trip\_Transportation**(trip\_transport\_sequence\_num, tripID, transportID, direction, price, booking\_status)

**Payments**(paymentID, tripID, userID, amount, payment\_method, payment\_status, payment\_timestamp)

**Refunds**(refundID, paymentID, refund\_amount, refund\_reason, refund\_timestamp)

**Support\_Queries**(queryID, userID, assigned\_admin\_user\_ID, subject, message\_body, query\_status, created\_timestamp, resolved\_timestamp)

**Company\_Approval\_Reviews**(reviewID, company\_user\_ID, admin\_user\_ID, decision, review\_notes, review\_timestamp)

### Relationships:

**Is\_A\_Traveller**(User\_ID)

**Is\_A\_Company**(User\_ID)

**Is\_A\_Administrator**(User\_ID)

**Creates**(User\_ID, Trip\_ID)

**Posts\_Hotel**(Company\_User\_ID, Hotel\_ID)

**Posts\_Transportation**(Company\_User\_ID, Transport\_ID)

**Includes\_Hotel**(Trip\_ID, Hotel\_ID)

**Includes\_Transportation**(Trip\_ID, Transport\_ID)

**Makes\_Payment**(User\_ID, Payment\_ID)

**Pays\_For**(Trip\_ID, Payment\_ID)

**Receives\_Refund**(Payment\_ID, Refund\_ID)

**Submits**(User\_ID, Query\_ID)

**Handles**(Admin\_User\_ID, Query\_ID)

**Reviews\_Company/Creates**(Assigned\_Admin\_User\_ID, Review\_ID)

**Reviewed\_For/Receives**(Company\_User\_ID, Review\_ID)

# MySQL Workbench Tables

## Users:

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

Limit to 1000 rows

1 • use travelog;

2

3 • SELECT \* From Users;

4

5

Result Grid

|    | userID | email                        | username      | password   | account_created     | account_status |
|----|--------|------------------------------|---------------|------------|---------------------|----------------|
| 1  |        | meghana.indukuri@example.com | meghana_j     | hashpass1  | 2004-01-03 08:12:00 | active         |
| 2  |        | sadhana.indukuri@example.com | sadhana_j     | hashpass2  | 2004-01-04 09:05:00 | active         |
| 3  |        | david.baker@example.com      | david_baker   | hashpass3  | 2004-01-05 10:14:00 | active         |
| 4  |        | yann.lecun@example.com       | yann_lecun    | hashpass4  | 2004-01-06 07:48:00 | active         |
| 5  |        | ethel.cain@example.com       | ethel_cain    | hashpass5  | 2004-01-07 11:10:00 | active         |
| 6  |        | hazier.byrne@example.com     | hazier        | hashpass6  | 2004-01-08 12:26:00 | active         |
| 7  |        | andrew.ng@example.com        | andrew_ng     | hashpass7  | 2004-01-09 08:33:00 | active         |
| 8  |        | demis.hassabis@example.com   | demis_h       | hashpass8  | 2004-01-10 09:41:00 | active         |
| 9  |        | bill.gates@example.com       | bill_g        | hashpass9  | 2004-01-11 10:03:00 | active         |
| 10 |        | jeff.bezos@example.com       | jeff_b        | hashpass10 | 2004-01-12 06:57:00 | active         |
| 11 |        | marriott.biz@example.com     | marriott_ops  | hashpass11 | 2004-01-13 09:15:00 | active         |
| 12 |        | hilton.biz@example.com       | hilton_team   | hashpass12 | 2004-01-14 09:44:00 | active         |
| 13 |        | hyatt.biz@example.com        | hyatt_group   | hashpass13 | 2004-01-15 10:11:00 | active         |
| 14 |        | delta.biz@example.com        | delta_routes  | hashpass14 | 2004-01-16 11:08:00 | active         |
| 15 |        | southwest.biz@example.com    | southwest...  | hashpass15 | 2004-01-17 12:20:00 | active         |
| 16 |        | amtrak.biz@example.com       | amtrak_ser... | hashpass16 | 2004-01-18 08:50:00 | active         |
| 17 |        | united.biz@example.com       | united_travel | hashpass17 | 2004-01-19 07:36:00 | active         |
| 18 |        | airbnb.biz@example.com       | airbnb_hos... | hashpass18 | 2004-01-20 09:29:00 | active         |
| 19 |        | expedia.biz@example.com      | expedia_pa... | hashpass19 | 2004-01-21 10:47:00 | active         |
| 20 |        | booking.biz@example.com      | booking_team  | hashpass20 | 2004-01-22 11:32:00 | active         |
| 21 |        | vi.admin@example.com         | vi_admin      | hashpass21 | 2004-01-23 08:17:00 | active         |
| 22 |        | jinx.admin@example.com       | jinx_admin    | hashpass22 | 2004-01-24 09:21:00 | active         |
| 23 |        | caitlyn.admin@example.com    | caitlyn_admin | hashpass23 | 2004-01-25 10:18:00 | active         |
| 24 |        | ekko.admin@example.com       | ekko_admin    | hashpass24 | 2004-01-26 07:59:00 | active         |
| 25 |        | jayce.admin@example.com      | jayce_admin   | hashpass25 | 2004-01-27 11:40:00 | active         |

Users 21 x

Object Info Session

Output

Navigator: travelog - Schema Query 1 x travelog - Schema

Limit to 1000 rows

1 • use travelog;

2

3 • SELECT \* From Users;

4

5

Result Grid

|    | userID | email                         | username      | password   | account_created     | account_status |
|----|--------|-------------------------------|---------------|------------|---------------------|----------------|
| 7  |        | andrew.ng@example.com         | andrew_ng     | hashpass7  | 2004-01-09 08:33:00 | active         |
| 8  |        | demis.hassabis@example.com    | demis_h       | hashpass8  | 2004-01-10 09:41:00 | active         |
| 9  |        | bill.gates@example.com        | bill_g        | hashpass9  | 2004-01-11 10:03:00 | active         |
| 10 |        | jeff.bezos@example.com        | jeff_b        | hashpass10 | 2004-01-12 06:57:00 | active         |
| 11 |        | marriott.biz@example.com      | marriott_ops  | hashpass11 | 2004-01-13 09:15:00 | active         |
| 12 |        | hilton.biz@example.com        | hilton_team   | hashpass12 | 2004-01-14 09:44:00 | active         |
| 13 |        | hyatt.biz@example.com         | hyatt_group   | hashpass13 | 2004-01-15 10:11:00 | active         |
| 14 |        | delta.biz@example.com         | delta_routes  | hashpass14 | 2004-01-16 11:08:00 | active         |
| 15 |        | southwest.biz@example.com     | southwest...  | hashpass15 | 2004-01-17 12:20:00 | active         |
| 16 |        | amtrak.biz@example.com        | amtrak_ser... | hashpass16 | 2004-01-18 08:50:00 | active         |
| 17 |        | united.biz@example.com        | united_travel | hashpass17 | 2004-01-19 07:36:00 | active         |
| 18 |        | airbnb.biz@example.com        | airbnb_hos... | hashpass18 | 2004-01-20 09:29:00 | active         |
| 19 |        | expedia.biz@example.com       | expedia_pa... | hashpass19 | 2004-01-21 10:47:00 | active         |
| 20 |        | booking.biz@example.com       | booking_team  | hashpass20 | 2004-01-22 11:32:00 | active         |
| 21 |        | vi.admin@example.com          | vi_admin      | hashpass21 | 2004-01-23 08:17:00 | active         |
| 22 |        | jinx.admin@example.com        | jinx_admin    | hashpass22 | 2004-01-24 09:21:00 | active         |
| 23 |        | caitlyn.admin@example.com     | caitlyn_admin | hashpass23 | 2004-01-25 10:18:00 | active         |
| 24 |        | ekko.admin@example.com        | ekko_admin    | hashpass24 | 2004-01-26 07:59:00 | active         |
| 25 |        | jayce.admin@example.com       | jayce_admin   | hashpass25 | 2004-01-27 11:40:00 | active         |
| 26 |        | viktor.admin@example.com      | viktor_admin  | hashpass26 | 2004-01-28 12:02:00 | active         |
| 27 |        | mel.admin@example.com         | mel_admin     | hashpass27 | 2004-01-29 08:43:00 | active         |
| 28 |        | heimerdingr.admin@example.com | heimer_admin  | hashpass28 | 2004-01-30 09:52:00 | active         |
| 29 |        | sevika.admin@example.com      | sevika_admin  | hashpass29 | 2004-01-31 10:24:00 | active         |
| 30 |        | vander.admin@example.com      | vander_admin  | hashpass30 | 2004-02-01 11:06:00 | active         |

Users 21 x

Object Info Session

Output

## Travelers:

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Traveler;

4

5

Result Grid

| userID | firstName | lastName | current_location |
|--------|-----------|----------|------------------|
| 1      | Meghana   | Indukuri | San Jose         |
| 2      | Sadhana   | Indukuri | Fremont          |
| 3      | David     | Baker    | Seattle          |
| 4      | Yann      | LeCun    | New York         |
| 5      | Ethel     | Cain     | Nashville        |
| 6      | Hozier    | Byrne    | Dublin           |
| 7      | Andrew    | Ng       | Palo Alto        |
| 8      | Demis     | Hassabis | London           |
| 9      | Bill      | Gates    | Redmond          |
| 10     | Jeff      | Bezos    | Seattle          |
| * NULL | NULL      | NULL     | NULL             |

## Company:

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Company;

4

5

Result Grid

| userID | name                   | business_type  | approval_status | phone        | verification_date   |
|--------|------------------------|----------------|-----------------|--------------|---------------------|
| 11     | Marriott International | hotel          | approved        | 408-555-2011 | 2004-02-05 09:00:00 |
| 12     | Hilton Hotels          | hotel          | approved        | 408-555-2012 | 2004-02-06 09:00:00 |
| 13     | Hyatt Hotels           | hotel          | approved        | 408-555-2013 | 2004-02-07 09:00:00 |
| 14     | Delta Air Lines        | transportation | approved        | 408-555-2014 | 2004-02-08 09:00:00 |
| 15     | Southwest Airlines     | transportation | approved        | 408-555-2015 | 2004-02-09 09:00:00 |
| 16     | Amtrak                 | transportation | approved        | 408-555-2016 | 2004-02-10 09:00:00 |
| 17     | United Airlines        | transportation | approved        | 408-555-2017 | 2004-02-11 09:00:00 |
| 18     | Airbnb                 | hotel          | approved        | 408-555-2018 | 2004-02-12 09:00:00 |
| 19     | Expedia                | both           | approved        | 408-555-2019 | 2004-02-13 09:00:00 |
| 20     | Booking.com            | both           | approved        | 408-555-2020 | 2004-02-14 09:00:00 |
| * NULL | NULL                   | NULL           | NULL            | NULL         | NULL                |

**Admin:**

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog**
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

```
1 • use travelog;
2
3 • SELECT * From Admin;
4
5
```

Result Grid

|   | userID | admin_level |
|---|--------|-------------|
| ▶ | 21     | senior      |
|   | 22     | standard    |
|   | 23     | super       |
|   | 24     | standard    |
|   | 25     | senior      |
|   | 26     | standard    |
|   | 27     | super       |
|   | 28     | senior      |
|   | 29     | standard    |
|   | 30     | senior      |
| * | NULL   | NULL        |

**Trip:**

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog**
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Trip;

4

5

Result Grid

| tripID | userID | trip_name             | start_location | destination | start_date | end_date   | status    | time_created        |
|--------|--------|-----------------------|----------------|-------------|------------|------------|-----------|---------------------|
| 101    | 1      | spring break monterey | San Jose       | Monterey    | 2004-03-01 | 2004-03-05 | planned   | 2004-02-15 10:00:00 |
| 102    | 2      | la weekend trip       | Fremont        | Los Angeles | 2004-03-06 | 2004-03-09 | booked    | 2004-02-16 10:00:00 |
| 103    | 3      | boston conference     | Seattle        | Boston      | 2004-03-10 | 2004-03-14 | planned   | 2004-02-17 10:00:00 |
| 104    | 4      | chicago talk          | New York       | Chicago     | 2004-03-15 | 2004-03-18 | booked    | 2004-02-18 10:00:00 |
| 105    | 5      | austin visit          | Toronto        | Austin      | 2004-03-19 | 2004-03-23 | cancelled | 2004-02-19 10:00:00 |
| 106    | 6      | dc museum trip        | Princeton      | Washington  | 2004-03-24 | 2004-03-27 | planned   | 2004-02-20 10:00:00 |
| 107    | 7      | miami break           | Palo Alto      | Miami       | 2004-03-28 | 2004-04-01 | booked    | 2004-02-21 10:00:00 |
| 108    | 8      | denver getaway        | London         | Denver      | 2004-04-02 | 2004-04-06 | planned   | 2004-02-22 10:00:00 |
| 109    | 9      | vegas summit          | Redmond        | Las Vegas   | 2004-04-07 | 2004-04-10 | booked    | 2004-02-23 10:00:00 |
| 110    | 10     | phoenix escape        | Seattle        | Phoenix     | 2004-04-11 | 2004-04-15 | planned   | 2004-02-24 10:00:00 |

## Hotel\_Listing:

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog**
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Hotel\_Listing;

4

5

Result Grid

| hotelID | company_userID | hotel_name            | location    | description                                 | rating | price_per_night | availability | listing_status |
|---------|----------------|-----------------------|-------------|---------------------------------------------|--------|-----------------|--------------|----------------|
| 201     | 11             | Marriott Monterey Bay | Monterey    | nice ocean view rooms and easy parking      | 4.5    | 189.00          | 1            | active         |
| 202     | 12             | Hilton Downtown LA    | Los Angeles | close to downtown and the convention center | 4.6    | 215.00          | 1            | active         |
| 203     | 13             | Hyatt Boston Harbor   | Boston      | quiet stay near the water                   | 4.4    | 198.00          | 1            | active         |
| 204     | 18             | Airbnb Chicago Loft   | Chicago     | spacious loft with self check in            | 4.3    | 172.00          | 1            | active         |
| 205     | 20             | Booking Austin Suites | Austin      | good for longer stays and work trips        | 4.2    | 185.00          | 1            | active         |
| 206     | 19             | Expedia Capitol Rooms | Washington  | walkable area with breakfast included       | 4.5    | 205.00          | 1            | active         |
| 207     | 11             | Marriott Miami Beach  | Miami       | beachfront stay with pool access            | 4.8    | 255.00          | 1            | active         |
| 208     | 12             | Hilton Denver Center  | Denver      | mountain views and clean rooms              | 4.4    | 210.00          | 1            | active         |
| 209     | 13             | Hyatt Vegas Strip     | Las Vegas   | right near the main attractions             | 4.1    | 240.00          | 1            | active         |
| 210     | 18             | Airbnb Phoenix House  | Phoenix     | cozy place with patio and parking           | 4.0    | 168.00          | 1            | active         |

## Transport\_Listing:

The screenshot shows the MySQL Workbench environment. At the top, there's a title bar "MySQL Workbench". Below it, a menu bar includes File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. A toolbar with various icons follows. The main window has a tab labeled "Query 1 x" and another tab "travelog - Schema". On the left, a "SCHEMAS" sidebar lists databases like bar\_beer\_drinker, indukuri, ticketmonarch\_db, travelog (selected), Tables, Views, Stored Procedures, and Functions. The central editor displays SQL code:

```
1 • use travelog;  
2  
3 • SELECT * From Trip_Hotels;  
4  
5
```

The bottom section features a "Result Grid" header with options for Filter Rows, Edit, Export/Import, Wrap Cell Content, and Truncate. Below this is a table of results from the "Trip\_Hotels" table.

|   | trip_hotel_sequence_num | tripID | hotelID | check_in_date | check_out_date | booked_price_per_night | total_cost | booking_status |
|---|-------------------------|--------|---------|---------------|----------------|------------------------|------------|----------------|
| ▶ | 1                       | 101    | 201     | 2004-03-01    | 2004-03-05     | 189.00                 | 756.00     | planned        |
|   | 1                       | 102    | 202     | 2004-03-06    | 2004-03-09     | 215.00                 | 645.00     | booked         |
|   | 1                       | 103    | 203     | 2004-03-10    | 2004-03-14     | 198.00                 | 792.00     | planned        |
|   | 1                       | 104    | 204     | 2004-03-15    | 2004-03-18     | 172.00                 | 516.00     | booked         |
|   | 1                       | 105    | 205     | 2004-03-19    | 2004-03-23     | 185.00                 | 740.00     | canceled       |
|   | 1                       | 106    | 206     | 2004-03-24    | 2004-03-27     | 205.00                 | 615.00     | planned        |
|   | 1                       | 107    | 207     | 2004-03-28    | 2004-04-01     | 255.00                 | 1020.00    | booked         |
|   | 1                       | 108    | 208     | 2004-04-02    | 2004-04-06     | 210.00                 | 840.00     | planned        |
|   | 1                       | 109    | 209     | 2004-04-07    | 2004-04-10     | 240.00                 | 720.00     | booked         |
|   | 1                       | 110    | 210     | 2004-04-11    | 2004-04-15     | 168.00                 | 672.00     | planned        |
| * | NONE                    | NONE   | NONE    | NONE          | NONE           | NONE                   | NONE       | NONE           |

At the very bottom, a status bar indicates "No object selected".





## Payments

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Payments;

4

5

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

| paymentID | tripID | userID | amount  | payment_method | payment_status | payment_timestamp   |
|-----------|--------|--------|---------|----------------|----------------|---------------------|
| 401       | 101    | 1      | 876.00  | Visa           | pending        | 2004-02-25 11:00:00 |
| 402       | 102    | 2      | 803.00  | Mastercard     | paid           | 2004-02-26 11:00:00 |
| 403       | 103    | 3      | 1074.00 | Visa           | pending        | 2004-02-27 11:00:00 |
| 404       | 104    | 4      | 658.00  | PayPal         | paid           | 2004-02-28 11:00:00 |
| 405       | 105    | 5      | 1045.00 | Visa           | refunded       | 2004-02-29 11:00:00 |
| 406       | 106    | 6      | 689.00  | Debit          | pending        | 2004-03-01 11:00:00 |
| 407       | 107    | 7      | 1338.00 | Visa           | paid           | 2004-03-02 11:00:00 |
| 408       | 108    | 8      | 1288.00 | Wire Transfer  | pending        | 2004-03-03 11:00:00 |
| 409       | 109    | 9      | 912.00  | Amex           | paid           | 2004-03-04 11:00:00 |
| 410       | 110    | 10     | 884.00  | Debit          | pending        | 2004-03-05 11:00:00 |
| *         | NULL   | NULL   | NULL    | NULL           | NULL           | NULL                |

## Refunds

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Refunds;

4

5

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

| refundID | paymentID | refund_amount | refund_reason             | refund_timestamp    |
|----------|-----------|---------------|---------------------------|---------------------|
| 501      | 401       | 30.00         | small fare adjustment     | 2004-03-06 09:00:00 |
| 502      | 402       | 20.00         | promo price correction    | 2004-03-07 09:00:00 |
| 503      | 403       | 45.00         | service issue credit      | 2004-03-08 09:00:00 |
| 504      | 404       | 18.00         | hotel adjustment          | 2004-03-09 09:00:00 |
| 505      | 405       | 1045.00       | full cancellation refund  | 2004-03-10 09:00:00 |
| 506      | 406       | 15.00         | partial refund            | 2004-03-11 09:00:00 |
| 507      | 407       | 55.00         | delay compensation        | 2004-03-12 09:00:00 |
| 508      | 408       | 40.00         | itinerary update refund   | 2004-03-13 09:00:00 |
| 509      | 409       | 25.00         | billing correction        | 2004-03-14 09:00:00 |
| 510      | 410       | 35.00         | support issued adjustment | 2004-03-15 09:00:00 |
| *        | NULL      | NULL          | NULL                      | NULL                |

## Support\_Queries

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog**
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Support\_Queries;

4

5

Result Grid

| queryID | userID | assigned_admin_user_ID | subject               | message_body                                          | query_status | created_timestamp   | resolved_timestamp  |
|---------|--------|------------------------|-----------------------|-------------------------------------------------------|--------------|---------------------|---------------------|
| 601     | 1      | 21                     | check in question     | hi, can i check in a little early for monterey?       | resolved     | 2004-03-01 12:00:00 | 2004-03-02 08:00:00 |
| 602     | 2      | 23                     | payment went through? | just wanted to make sure my la booking is confr...    | resolved     | 2004-03-02 12:00:00 | 2004-03-03 08:00:00 |
| 603     | 3      | 25                     | room dates look off   | my boston stay dates look a day short on the p...     | in_progress  | 2004-03-03 12:00:00 | NULL                |
| 604     | 4      | 24                     | train timing question | the arrival time on my chicago trip looks confusing   | resolved     | 2004-03-04 12:00:00 | 2004-03-05 08:00:00 |
| 605     | 5      | 22                     | refund amount         | can you tell me what my austin cancellation refu...   | closed       | 2004-03-05 12:00:00 | 2004-03-06 08:00:00 |
| 606     | 6      | 26                     | bus details           | is there a way to change the bus time for dc?         | open         | 2004-03-06 12:00:00 | NULL                |
| 607     | 7      | 27                     | hotel breakfast       | does the miami hotel include breakfast or not?        | in_progress  | 2004-03-07 12:00:00 | NULL                |
| 608     | 8      | 28                     | flight info           | i need help checking my denver itinerary again        | open         | 2004-03-08 12:00:00 | NULL                |
| 609     | 9      | 29                     | trip total            | can someone resend my vegas trip summary?             | resolved     | 2004-03-09 12:00:00 | 2004-03-10 08:00:00 |
| 610     | 10     | 30                     | route switch          | is it possible to switch the phoenix transport opt... | open         | 2004-03-10 12:00:00 | NULL                |
| 61039   | NULL   | NULL                   | NULL                  | NULL                                                  | NULL         | NULL                | NULL                |

## Company\_Approval\_Reviews

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog**
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Company\_Approval\_Reviews;

4

5

Result Grid

| reviewID | company_user_ID | admin_user_ID | decision | review_notes                            | review_timestamp    |
|----------|-----------------|---------------|----------|-----------------------------------------|---------------------|
| 701      | 11              | 21            | approved | marriott documents checked and approved | 2004-02-15 13:00:00 |
| 702      | 12              | 23            | approved | hilton business profile looks good      | 2004-02-16 13:00:00 |
| 703      | 13              | 25            | approved | hyatt verification completed            | 2004-02-17 13:00:00 |
| 704      | 14              | 24            | approved | delta account approved after review     | 2004-02-18 13:00:00 |
| 705      | 15              | 22            | approved | southwest submission approved           | 2004-02-19 13:00:00 |
| 706      | 16              | 26            | approved | amtrak profile verified                 | 2004-02-20 13:00:00 |
| 707      | 17              | 27            | approved | united documents matched                | 2004-02-21 13:00:00 |
| 708      | 18              | 28            | approved | airbnb hosting account approved         | 2004-02-22 13:00:00 |
| 709      | 19              | 29            | approved | expedia partner account approved        | 2004-02-23 13:00:00 |
| 710      | 20              | 30            | approved | booking account approved successfully   | 2004-02-24 13:00:00 |
| NULL     | NULL            | NULL          | NULL     | NULL                                    | NULL                |

## Is\_A\_Traveller

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog**
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Is\_A\_Traveller;

4

5

Result Grid

| userID |
|--------|
| 1      |
| 2      |
| 3      |
| 4      |
| 5      |
| 6      |
| 7      |
| 8      |
| 9      |
| 10     |
| NULL   |

## Is\_A\_Company

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- bar\_beer\_drinker
- indukuri
- ticketmonarch\_db
- travelog**
  - Tables
  - Views
  - Stored Procedures
  - Functions

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Is\_A\_Company;

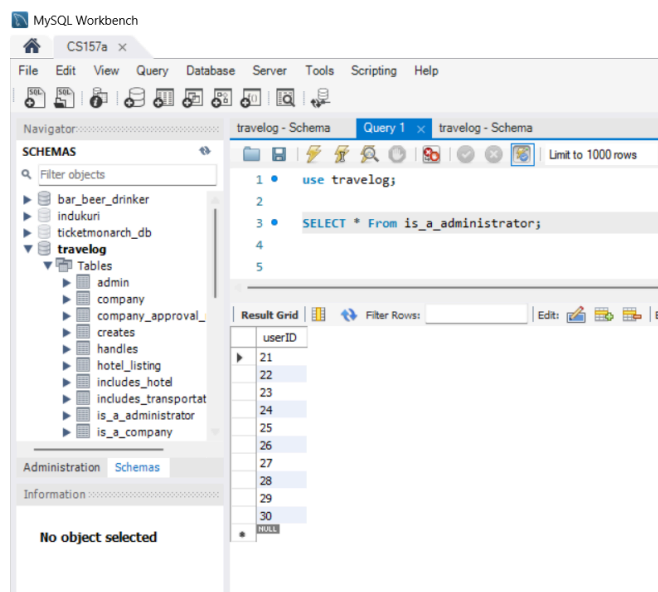
4

5

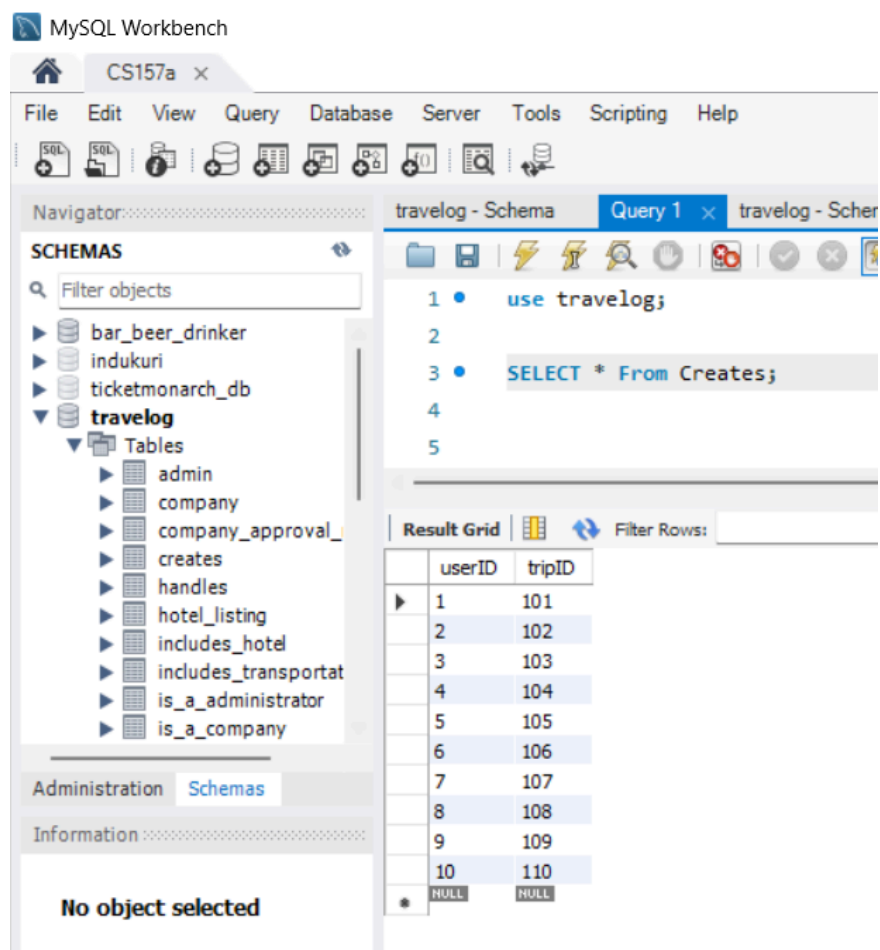
Result Grid

| userID |
|--------|
| 11     |
| 12     |
| 13     |
| 14     |
| 15     |
| 16     |
| 17     |
| 18     |
| 19     |
| 20     |
| NULL   |

## Is\_A\_Administrator



## Creates



## Posts\_Hotel

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Posts\_Hotel;

4

5

Result Grid

| company_userID | hotelID |
|----------------|---------|
| 11             | 201     |
| 12             | 202     |
| 13             | 203     |
| 18             | 204     |
| 20             | 205     |
| 19             | 206     |
| 11             | 207     |
| 12             | 208     |
| 13             | 209     |
| 18             | 210     |
| NULL           | NULL    |

## Posts\_Transportation

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT \* From Posts\_Transportation;

4

5

Result Grid

| company_userID | transportID |
|----------------|-------------|
| 14             | 301         |
| 15             | 302         |
| 17             | 303         |
| 16             | 304         |
| 14             | 305         |
| 16             | 306         |
| 17             | 307         |
| 15             | 308         |
| 14             | 309         |
| 16             | 310         |
| NULL           | NULL        |

## Includes\_Hotel

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Includes_Hotel;
4
5

```

Result Grid

| tripID | hotelID |
|--------|---------|
| 101    | 201     |
| 102    | 202     |
| 103    | 203     |
| 104    | 204     |
| 105    | 205     |
| 106    | 206     |
| 107    | 207     |
| 108    | 208     |
| 109    | 209     |
| 110    | 210     |
| NULL   | NULL    |

## Includes\_Transportation

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Includes_Transportation;
4
5

```

Result Grid

| tripID | transportID |
|--------|-------------|
| 101    | 301         |
| 102    | 302         |
| 103    | 303         |
| 104    | 304         |
| 105    | 305         |
| 106    | 306         |
| 107    | 307         |
| 108    | 308         |
| 109    | 309         |
| 110    | 310         |
| NULL   | NULL        |

## Makes\_Payment

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Makes_Payment;
4
5

```

Result Grid

|    | userID | paymentID |
|----|--------|-----------|
| 1  | 401    |           |
| 2  | 402    |           |
| 3  | 403    |           |
| 4  | 404    |           |
| 5  | 405    |           |
| 6  | 406    |           |
| 7  | 407    |           |
| 8  | 408    |           |
| 9  | 409    |           |
| 10 | 410    |           |
| *  | NULL   | NULL      |

## Pays\_For

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Pays_For;
4
5

```

Result Grid

|     | tripID | paymentID |
|-----|--------|-----------|
| 101 | 401    |           |
| 102 | 402    |           |
| 103 | 403    |           |
| 104 | 404    |           |
| 105 | 405    |           |
| 106 | 406    |           |
| 107 | 407    |           |
| 108 | 408    |           |
| 109 | 409    |           |
| 110 | 410    |           |
| *   | NULL   | NULL      |



## Recieves\_Refund

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

**Table: receives\_refund**

**Columns:**  
paymentID int PK

1 • use travelog;

2

3 • SELECT \* From receives\_refund;

4

5

Result Grid Filter Rows: Edit:

| paymentID | refundID |
|-----------|----------|
| 401       | 501      |
| 402       | 502      |
| 403       | 503      |
| 404       | 504      |
| 405       | 505      |
| 406       | 506      |
| 407       | 507      |
| 408       | 508      |
| 409       | 509      |
| 410       | 510      |
| NULL      | NULL     |

## Submits

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Sch

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

**Table: receives\_refund**

**Columns:**  
paymentID int PK

1 • use travelog;

2

3 • SELECT \* From Submits;

4

5

Result Grid Filter Rows: Edit:

| userID | queryID |
|--------|---------|
| 1      | 601     |
| 2      | 602     |
| 3      | 603     |
| 4      | 604     |
| 5      | 605     |
| 6      | 606     |
| 7      | 607     |
| 8      | 608     |
| 9      | 609     |
| 10     | 610     |
| NULL   | NULL    |

## Handles

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

Table: **receives\_refund**

Columns:

admin\_user\_ID int(11) queryID int(11)

```

1 • use travelog;
2
3 • SELECT * From Handles;
4
5

```

Result Grid

| admin_user_ID | queryID |
|---------------|---------|
| 21            | 601     |
| 23            | 602     |
| 25            | 603     |
| 24            | 604     |
| 22            | 605     |
| 26            | 606     |
| 27            | 607     |
| 28            | 608     |
| 29            | 609     |
| 30            | 610     |
| NULL          | NULL    |

## Reviews\_Company

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

**SCHEMAS**

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

Table: **reviews\_company**

Columns:

admin\_user\_ID int(11) reviewID int(11)

```

1 • use travelog;
2
3 • SELECT * From reviews_company;
4
5

```

Result Grid

| admin_user_ID | reviewID |
|---------------|----------|
| 21            | 701      |
| 23            | 702      |
| 25            | 703      |
| 24            | 704      |
| 22            | 705      |
| 26            | 706      |
| 27            | 707      |
| 28            | 708      |
| 29            | 709      |
| 30            | 710      |
| NULL          | NULL     |

## Reviewed\_For

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is\_a\_administrator
- is\_a\_company
- is\_a\_traveller
- makes\_payment
- payments
- pays\_for
- posts\_hotel
- posts\_transportation
- receives\_refund
- refunds
- reviewed\_for
- reviews\_company
- submits
- support\_queries
- transport\_listing

Administration Schemas

Information

Table: reviews\_company

Columns:

1 • use travelog;

2

3 • SELECT \* From Reviewed\_For;

4

5

Result Grid

|    | company_user_ID | reviewID |
|----|-----------------|----------|
| 11 |                 | 701      |
| 12 |                 | 702      |
| 13 |                 | 703      |
| 14 |                 | 704      |
| 15 |                 | 705      |
| 16 |                 | 706      |
| 17 |                 | 707      |
| 18 |                 | 708      |
| 19 |                 | 709      |
| 20 |                 | 710      |
| *  | NULL            | NULL     |

## Project Implementation

### Setup:

1. Use the sql/setup.sql in the workbench to create the schema and tables needed if not already done. (a) To do this use `mysql -u root -p` from terminal to get MySQL terminal (b) Run `SOURCE C:/path-to/setup.sql`;
2. in WEB-INF make a copy of the `db_config_example.jsp` and fill it out with your information

▼ WEB-INF

 db\_config\_example.jsp

 db\_config.jsp

```
<%!
// copy this file into db_config.jsp

static final String DB_URL    = "jdbc:mysql://localhost:3306/travelog";
static final String DB_USER   = "root";
static final String DB_PASS   = "root"; // Your MySQL password
static final String DB_DRIVER = "com.mysql.jdbc.Driver";

// SerpAPI (Google Flights)
static final String SERPAPI_KEY = "YOUR_SERPAPI_KEY_HERE";
%>
```

3. mysql connecter should be in tomcat lib folder
4. json.jar also needs to be in tomcat lib folder (its used for parsing the flight API response) download from <https://repo1.maven.org/maven2/org/json/json/20240303/json-20240303.jar> and put it in tomcat/lib

▼ lib

 json.jar

 mysql-connector-j-9.6.0.jar

5. Tomcat's Root should point to this (so in tomcat/conf/Catalina/localhost create file `ROOT.xml` and copy paste this `docBase="path to folder here" reloadable="true"/>)`

▼ conf

▼ Catalina

▼ localhost

 ROOT.xml

6. Start tomcat and go to <http://localhost:8080/>
7. For flight API, we are using SERPAPI, add your own free api key into the db\_config and should work immediately when searching flights

### Implementation:

For our web application, we implement using JSP and a MySQL database. Below we go over each functional requirement and how it maps to our JSP files which deal with GUI and database interaction.

## 1. Traveler View

### a. Register Account

Files: register.jsp, register\_process.jsp

Tables Affected: Users, Traveler, Is\_A\_Traveller

SQL Operations:

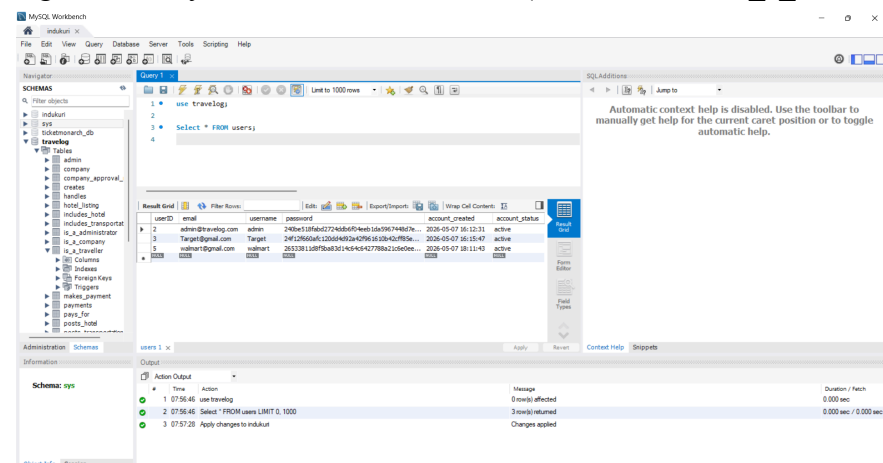
- SELECT (duplicate username or email check)
- INSERT into Users (with SHA-256 hashes password)
- INSERT into Traveler (subclass record)
- INSERT into Is\_A\_Traveller (relation table)

Implementation Process:

- (1) Form fields read from request
  - (a) Validation: required fields, correct email format, and proper password length
- (2) Passwords are hashed using SHA-256
- (3) Duplicate check for users
- (4) Insert into users and get the generated userID
- (5) Insert into traveller and is\_a\_traveller with that userID
- (6) Redirect to login.jsp with a message showing success

Figures:

Figure 1.1: MySQL Workbench Before (users, traveler, is\_a traveller)



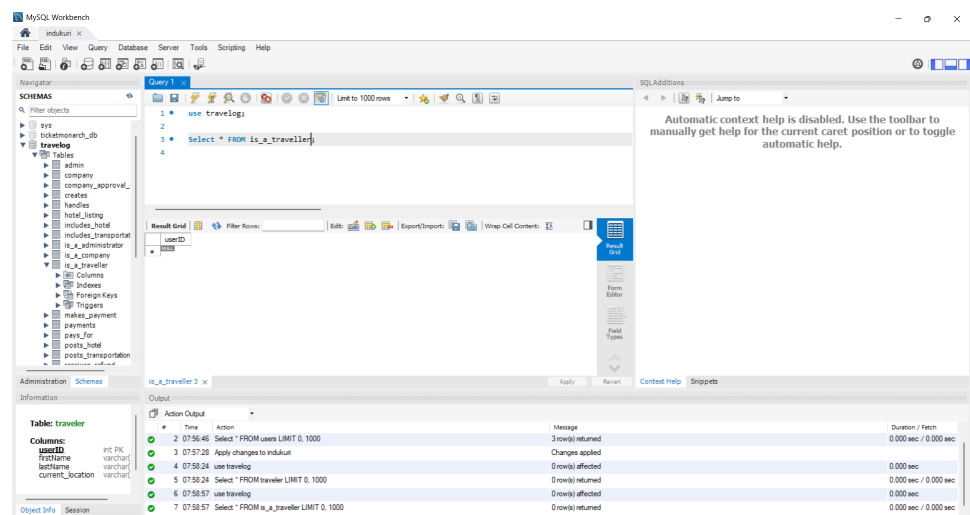
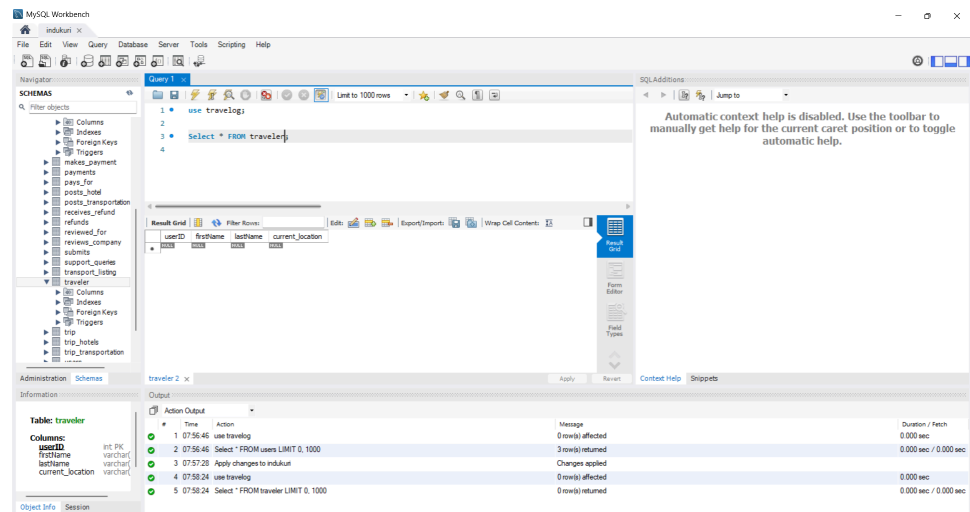


Figure 1.2: Browser UI with filled Create Account Fields

Get started.  
Create your Traveling account

USERNAME  
meghana123

EMAIL  
meghana.indukuri@gmail.com

PASSWORD  
\*\*\*\*\*

ACCOUNT TYPE  
Traveler

FIRST NAME  
Meghana

LAST NAME  
Indukuri

HOME LOCATION  
Cupertino

Create Account

or

Already have an account? Sign In

Figure 1.3: Browser UI after Clicking Create Account

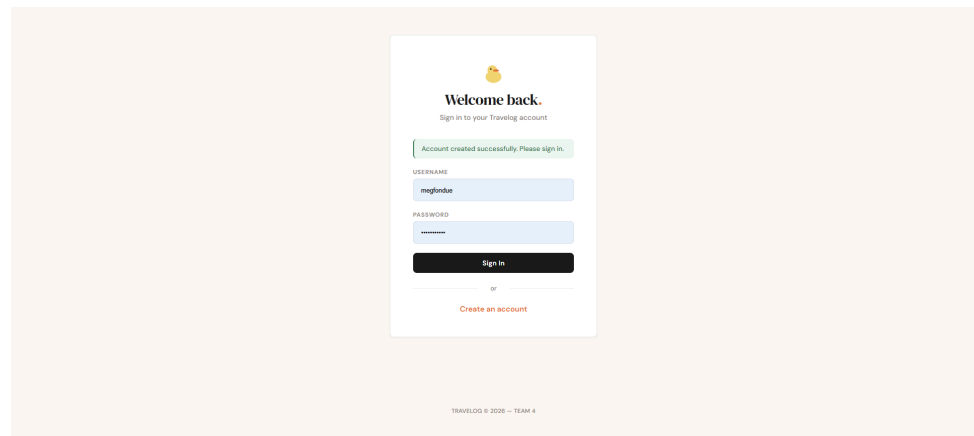
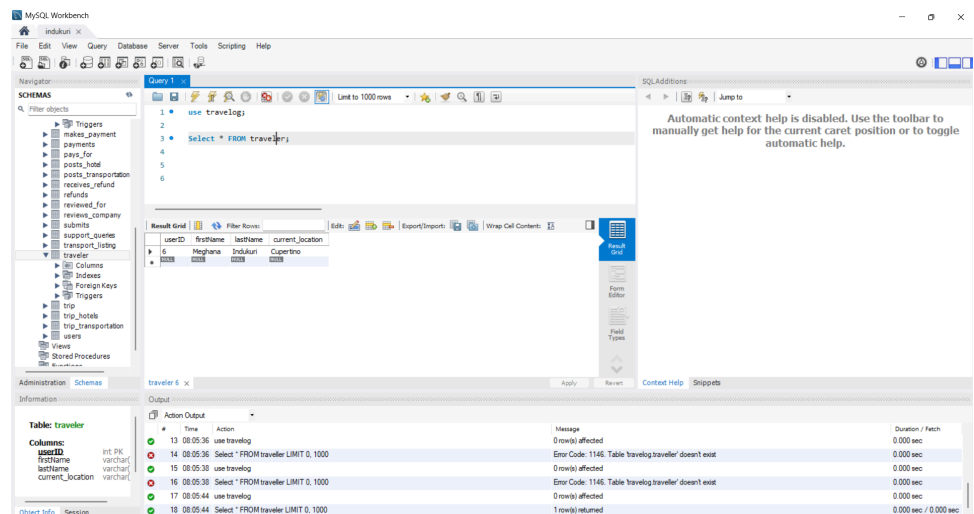
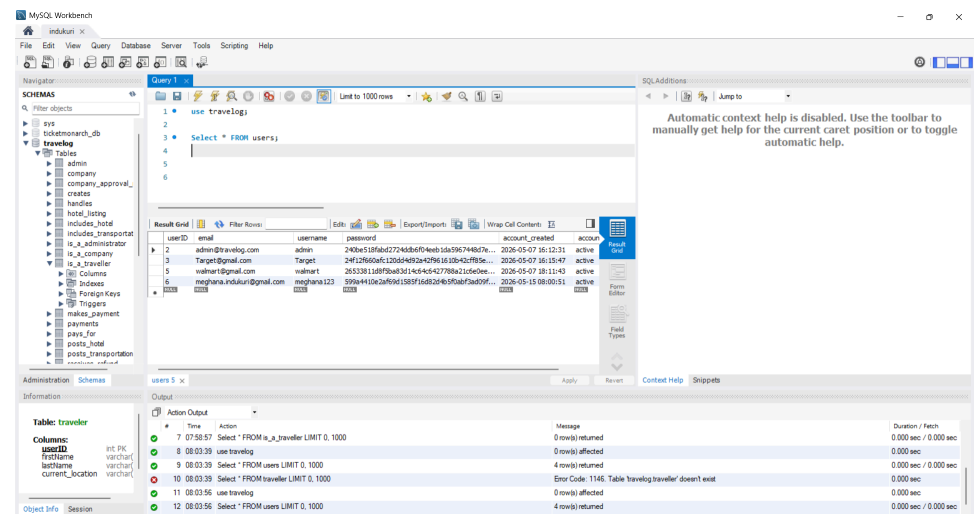
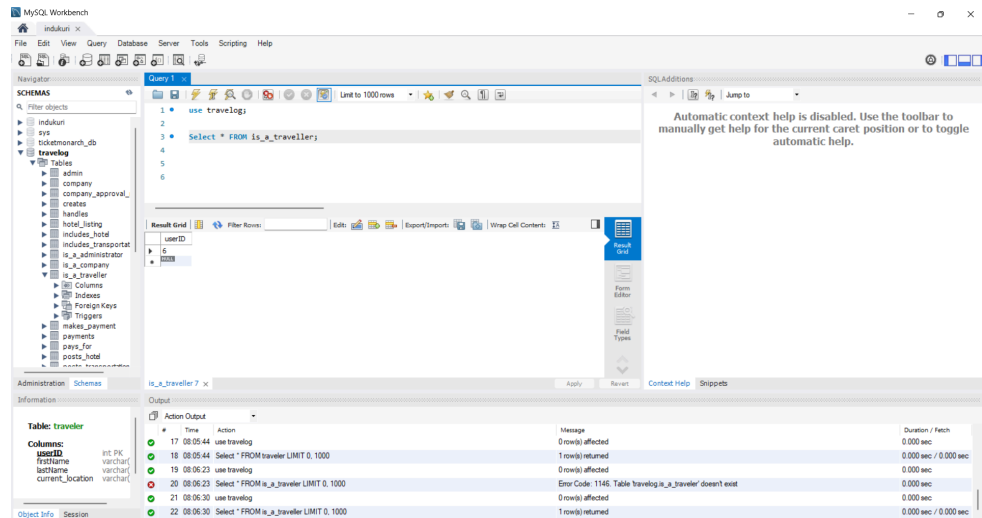


Figure 1.4: MySQL Workbench After (users, traveler, is\_a\_traveller)





## b. Account Sign In

Files: login.jsp, login\_process.jsp

Tables Affected: Users joined with Traveler, Admin, or Company (sequentially)

SQL Operations:

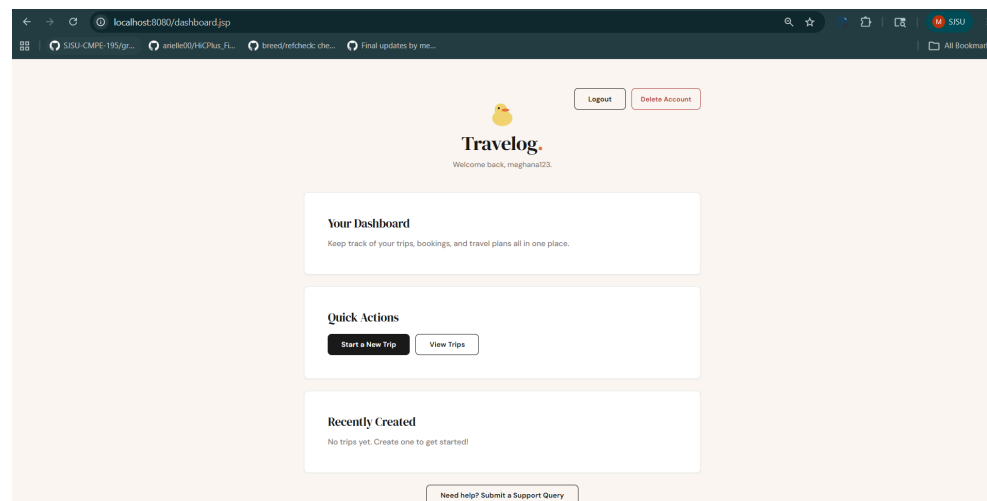
- SELECT (worst case three times: traveler, admin, or company)

Implementation Process:

- (1) Hash the password inputted by user
- (2) Try match with Users JOIN Traveler
- (3) If no match, try match with Users JOIN Admin
- (4) If no match, try match with Users JOIN Company
- (5) Match means session information stored and redirected to dashboard

Figures:

Figure 2.1: Browser UI for Login Showing Signed In





### c. Account Logout

Files: logout.jsp

Tables Affected: none

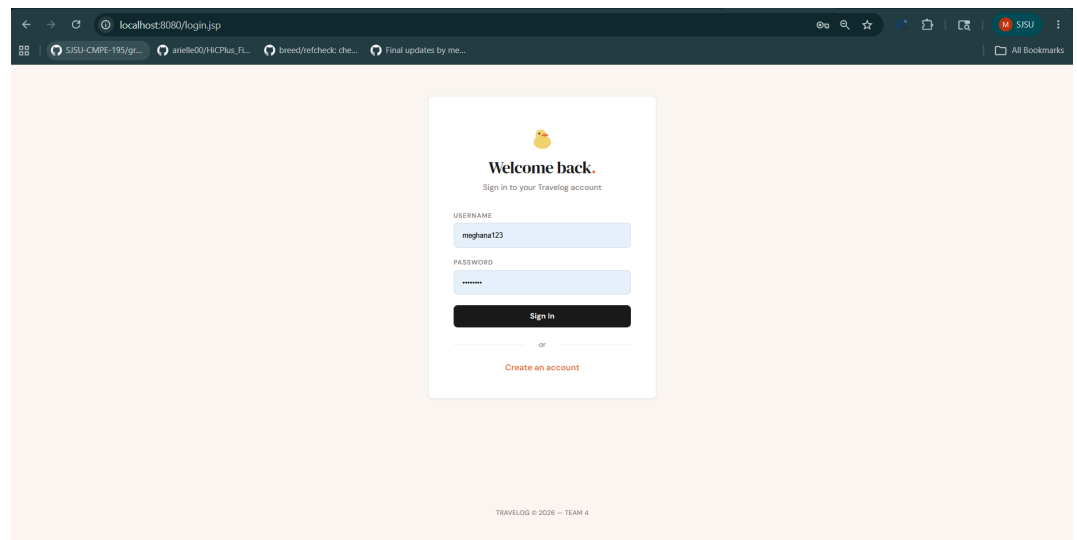
SQL Operations: none

Implementation Process:

- Clear the session data
- Redirect to login.jsp

Figures:

Figure 3.1: Login JSP redirect after Logout



### d. Delete Account

Files: delete\_account.jsp, delete\_account\_process.jsp

Tables Affected: Users which cascades to Traveler, Trip, Payments and more.

SQL Operations:

- SELECT (for password verification process)
- DELETE from Users

Implementation Process:

- Hash the user submitted password and check to get current userID
- Delete from users, we use foreign-key cascade which helps removes all the other rows
- Clear the session and redirect to login.jsp

Figures:

Figure 4.1: Users and is\_a\_traveller Tables Before Delete

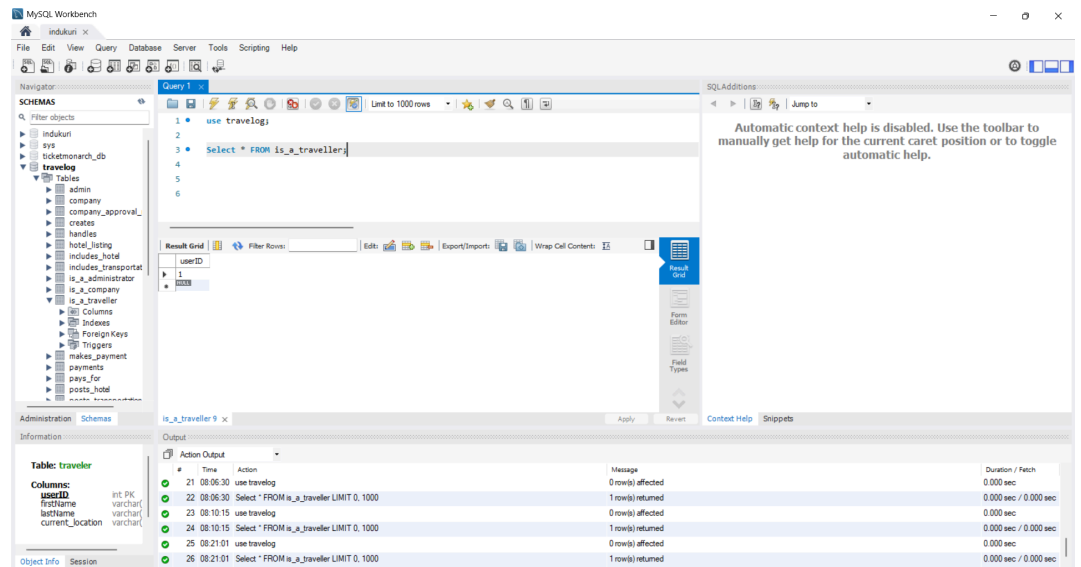
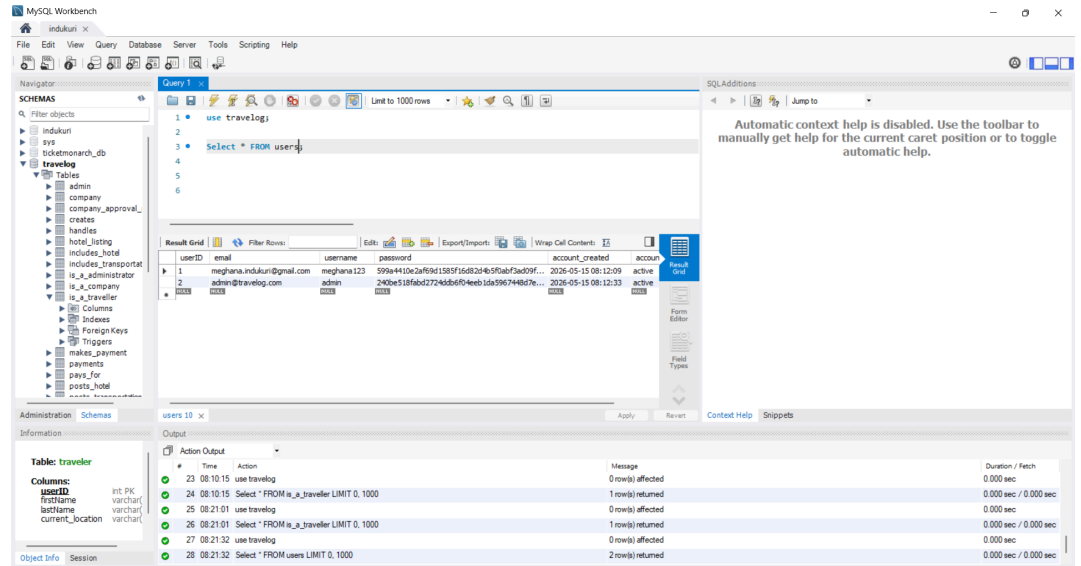


Figure 4.2: Delete UI on Browser

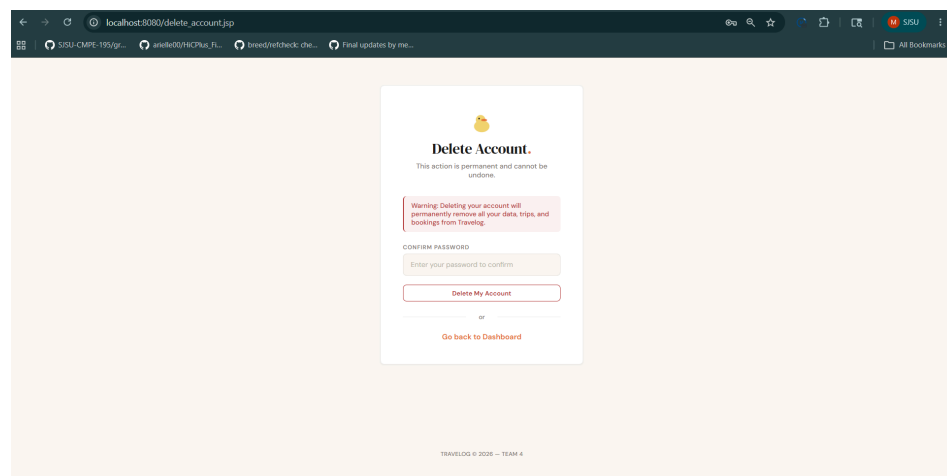
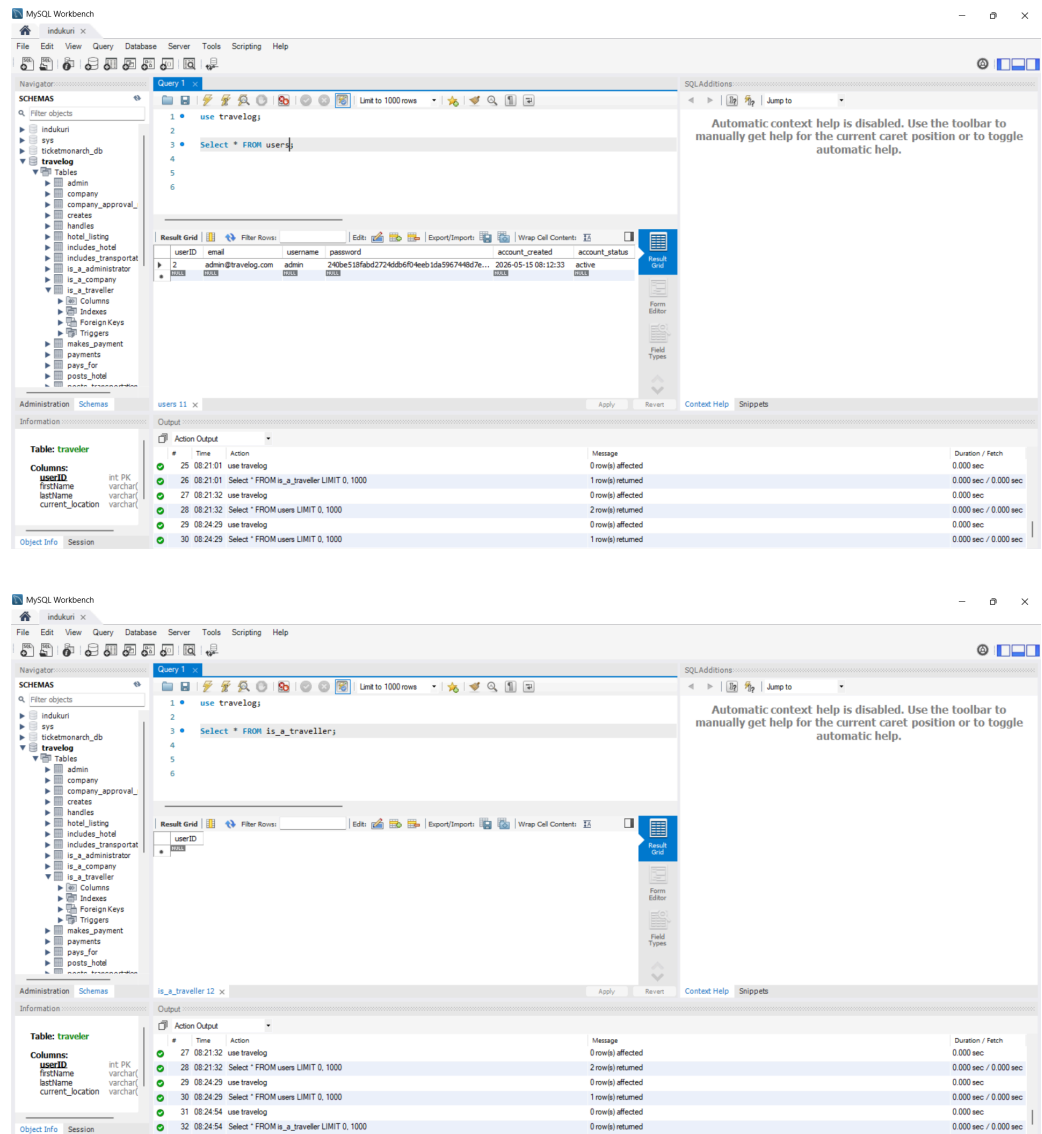


Figure 4.4: Workbench After Delete



## 2. Trip Management:

### a. View Trips

Files: view\_trips.jsp, dashboard.jsp

Tables Affected: Users, Traveler, Trip

SQL Operations:

- SELECT

Implementation Process:

- Get userID from the session
- Join Users, Traveler, Trip (on the userID)

- Show the results on the dashboard
- Had to use cool methodology with ORDER BY FIELD to order based on status: planned, booked, cancelled. Link for resource used: <https://stackoverflow.com/questions/14104055/ordering-by-specific-field-value-first>

Figures:

Figure 5.1: Browser Dashboard with Trips

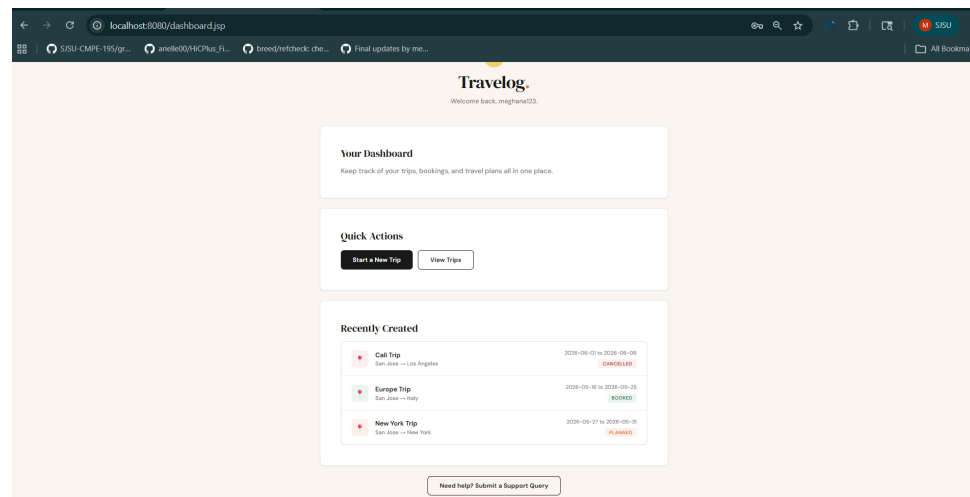


Figure 5.2: Browser (view trips.jsp) Showing All Trips

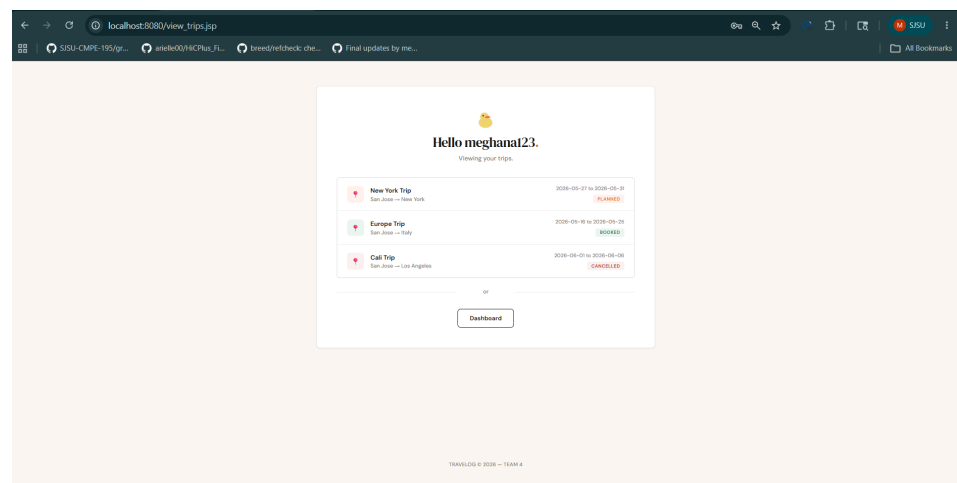
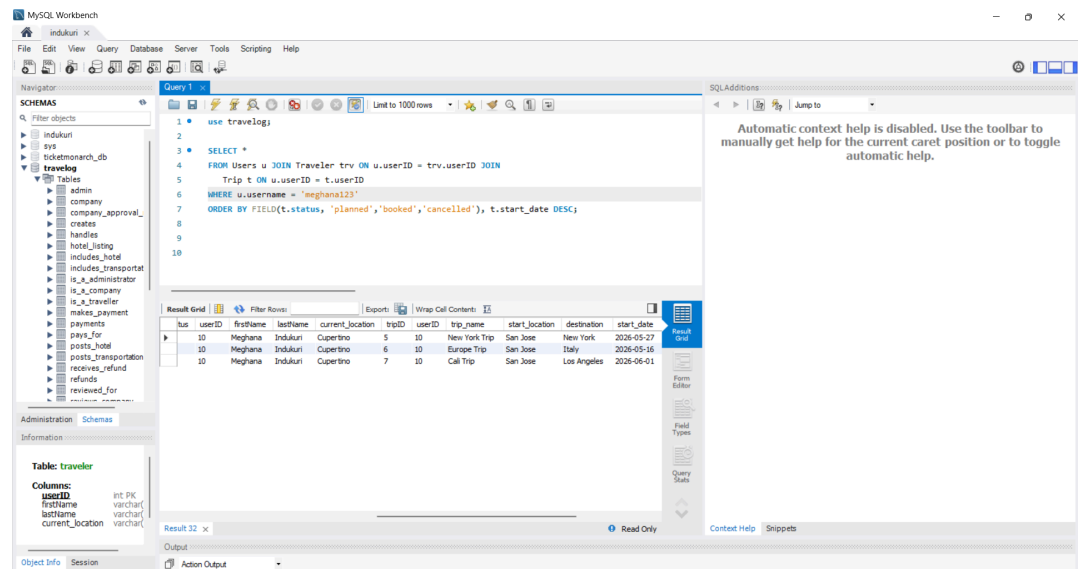


Figure 5.3: Workbench (JSP query run manually), query on username which is unique and do a three-table join for all information



## b. Create Trip

Files: create\_trips.js, create\_trip\_process.js

Tables Affected: Trip, Creates

SQL Operations:

- Insert into Trip
- Insert into Creates (relation table)

Implementation Process:

- Validate the name and dates (like start day can't be before current day)
- Insert into Trip with the status set to planned
- Get the generated tripID, and put it into Creates
- Redirect to trip\_details.js

Figures:

Figure 6.1: Workbench Before Create Trip

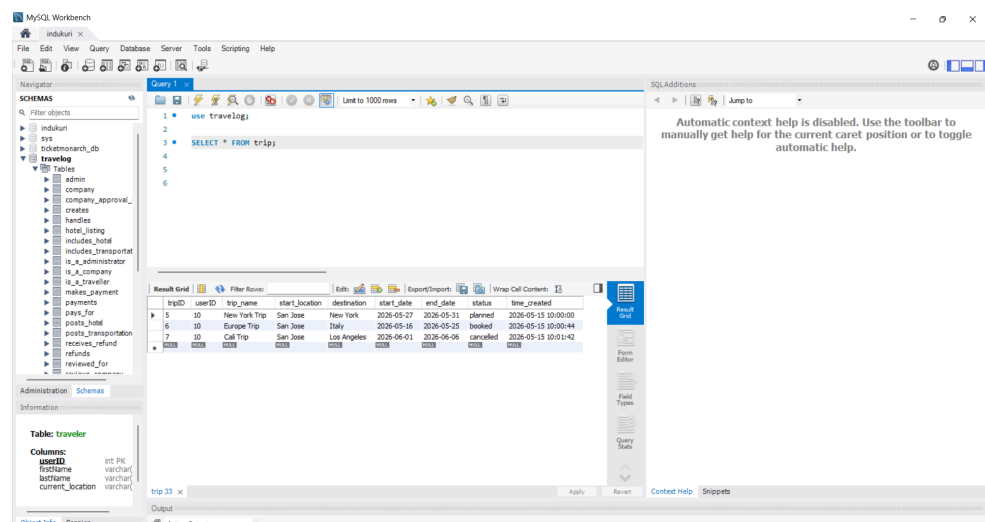


Figure 6.2: Browser Create Trip Form Filled Out

localhost/create\_trip.jsp

Create Your Trip.

TRIP NAME  
Iceland Trip

CURRENT LOCATION  
Cupertino

DESTINATION  
Reykjavik

START DATE  
05/16/2026

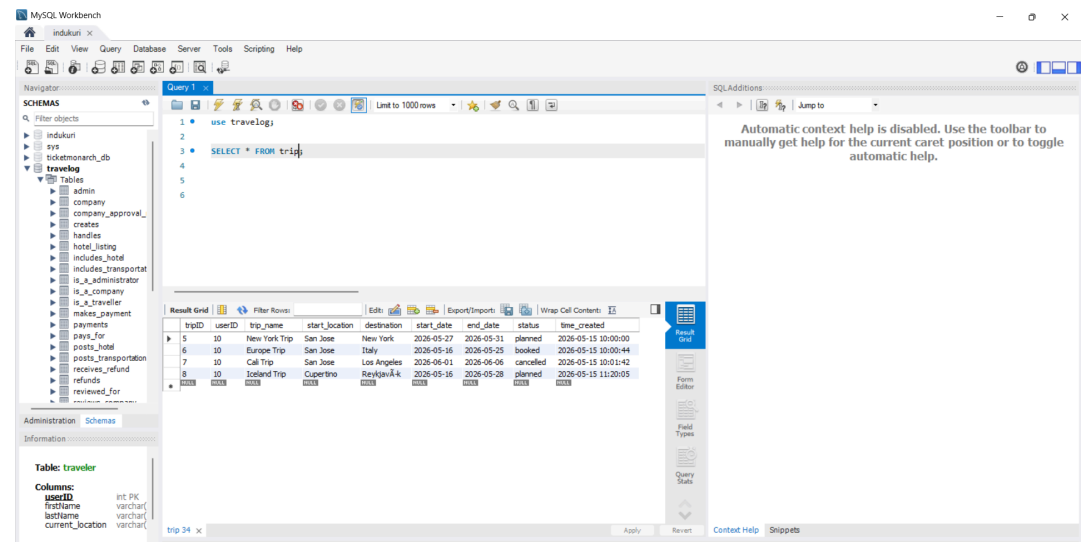
END DATE  
05/28/2026

Create Trip

or

[Go to Traveling Dashboard](#)

Figure 6.3: Workbench After Create Trip



### c. Select Starting and Final Destinations

Files: create\_trips.jsp, trip\_details.jsp, edit\_trip\_process.jsp

Tables Affected: Trip

SQL Operations:

- UPDATE Trip

Implementation Process:

- User enters start location and the destination
- Then Trip updated based on the location information

Figures:

Figure 7.1: Workbench Before with Old Values

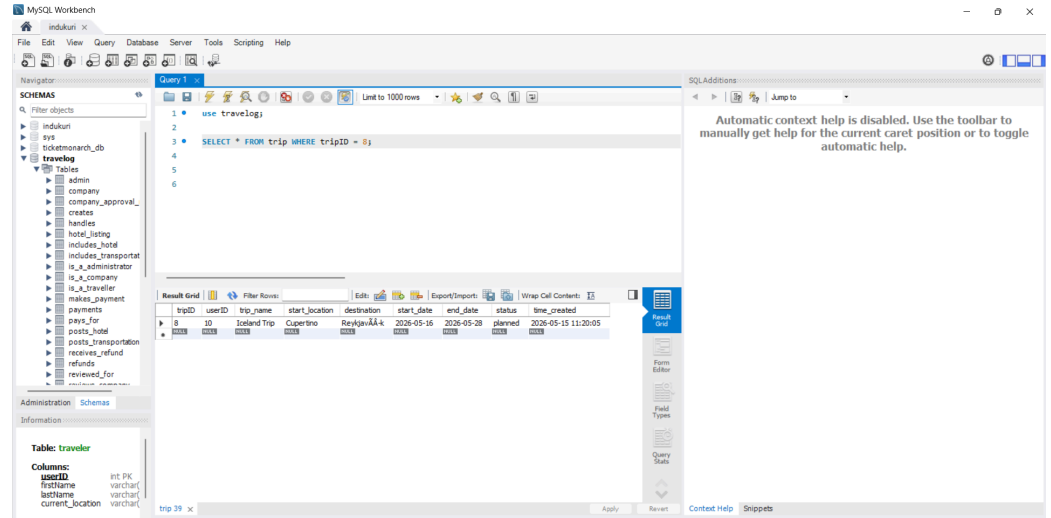


Figure 7.2: Browser with Changes to Destinations

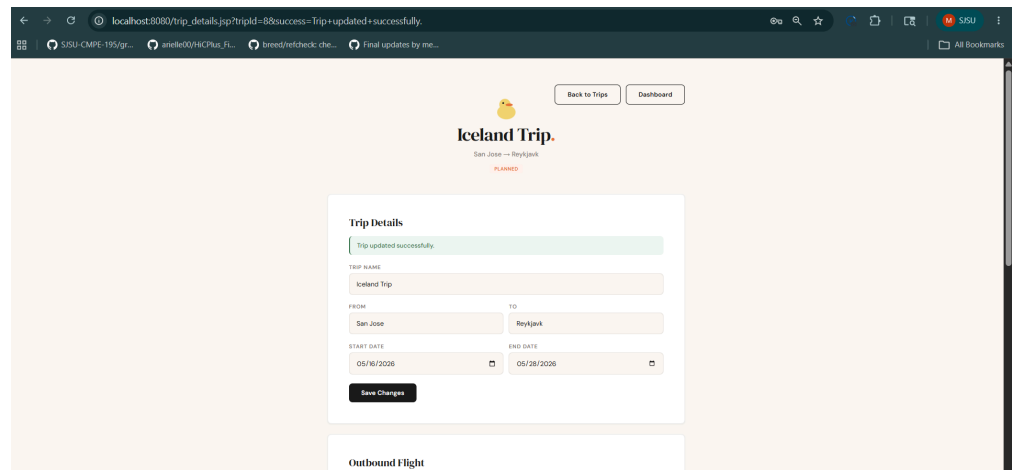
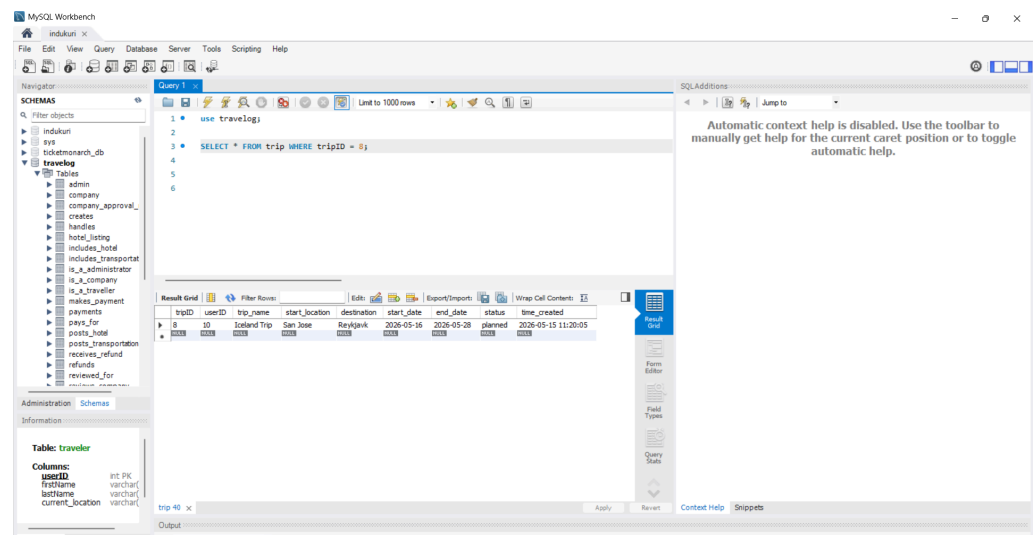


Figure 7.3: Workbench After



#### d. Select Preferred Dates

Files: create\_trips.jsp, edit\_trip\_process.jsp

Tables Affected: Trip

SQL Operations:

- UPDATE Trip

Implementation Process:

- Validate the dates for the trip
- Then Trip updated based on the dates information

Figures:

Figure 8.1: Workbench Before with Old Values

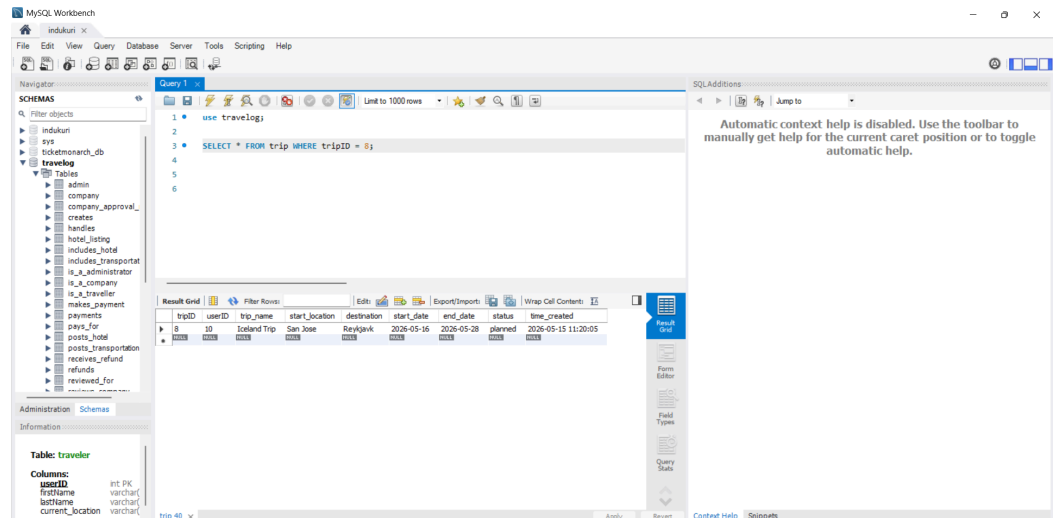


Figure 8.2: Browser with Changes to Dates

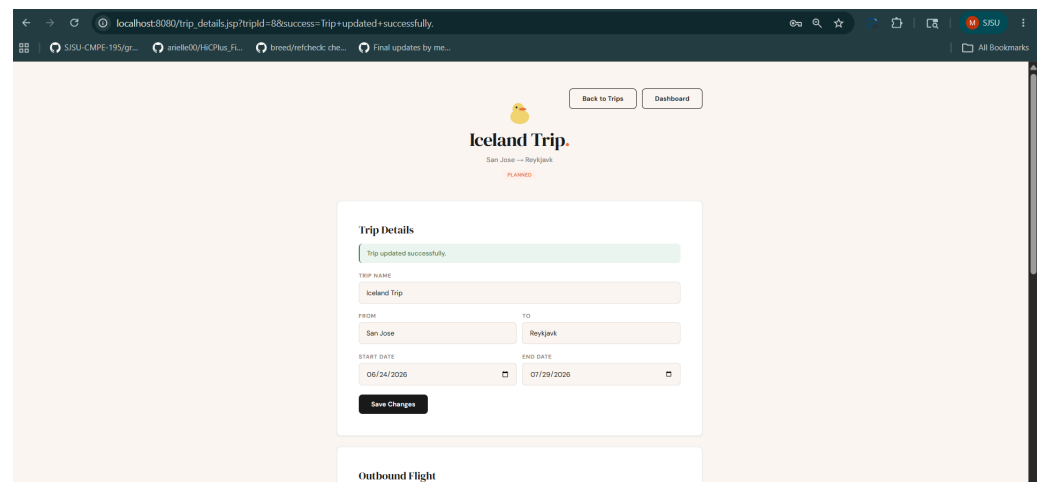
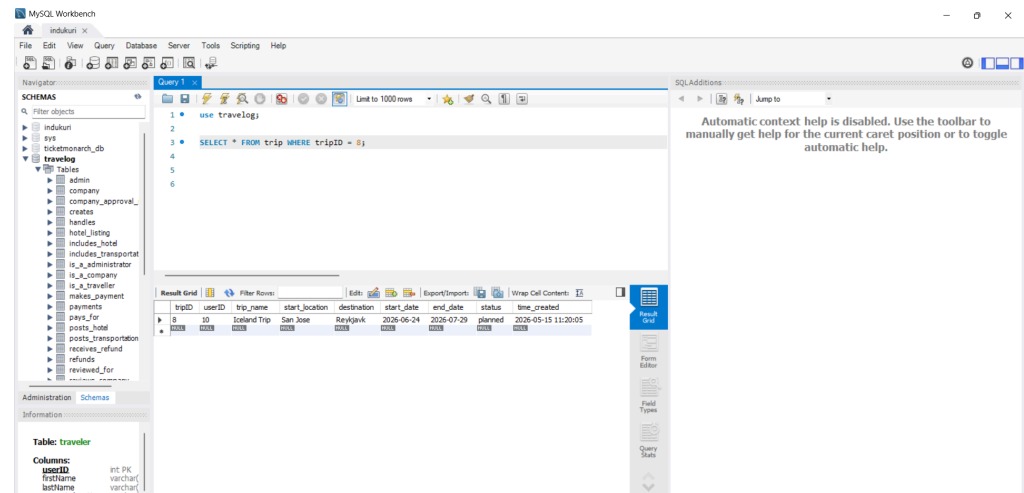




Figure 8.3: Workbench After



#### e. Browse and Filter Transport Options

Files: browse\_transportation.jsp

Tables Affected: Transport\_Listing

SQL Operations:

- SELECT (cache checking, which I chose 30 days for)
- INSERT (cache API results)
- DELETE (delete old/stale/outdated rows)

Implementation Process:

- Read in the search parameters from user (like departure\_id, arrival\_id, and outbound date)
- Check the transport\_listing currency cache
- If not there, call SerpAPI Google Flights:  
[https://serpapi.com/google-flights-api?gad\\_source=1&gad\\_campaignid=1061187028&gbraid=0AAAAADD8kqO26vnVRkX5b4TOoc-ScaApr&gclid=Cj0KCQjwiJvQBhCYARIsAMjts3L1u1fVx7Wi6k95\\_P69sv45NY7vgc1K90q0G62wp0xBwIJmVQN8HYwaAlZzEALw\\_wcB](https://serpapi.com/google-flights-api?gad_source=1&gad_campaignid=1061187028&gbraid=0AAAAADD8kqO26vnVRkX5b4TOoc-ScaApr&gclid=Cj0KCQjwiJvQBhCYARIsAMjts3L1u1fVx7Wi6k95_P69sv45NY7vgc1K90q0G62wp0xBwIJmVQN8HYwaAlZzEALw_wcB)
- Parse JSON, Insert into Transport Listing
- Render the results, use Javascript to show the price and time filtering (for efficiency)

Figures:

Figure 9.1: Workbench Query Before SerpAPI call

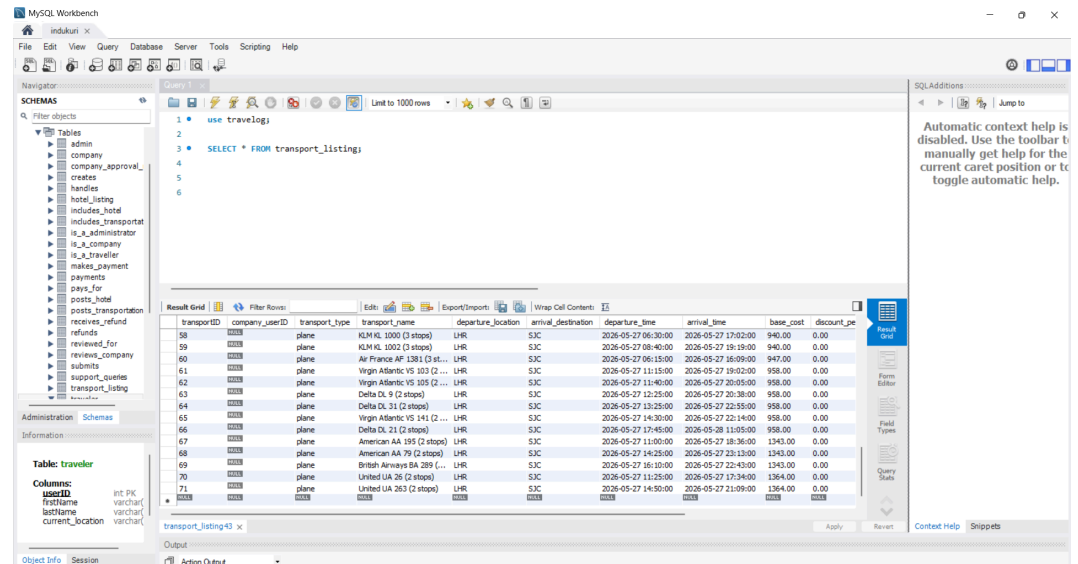


Figure 9.2: Browser Transportation Search Form Filled Out

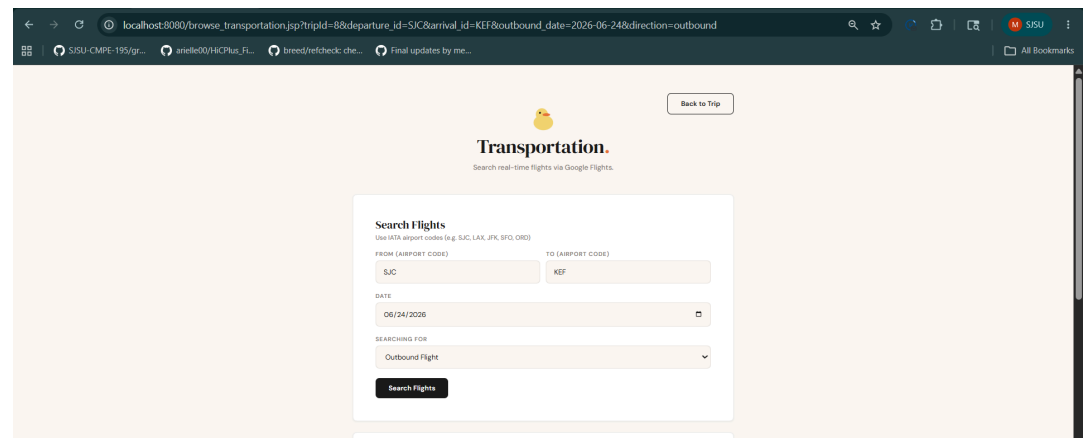


Figure 9.3: Workbench Query to Show Cached Changes from SerpAPI

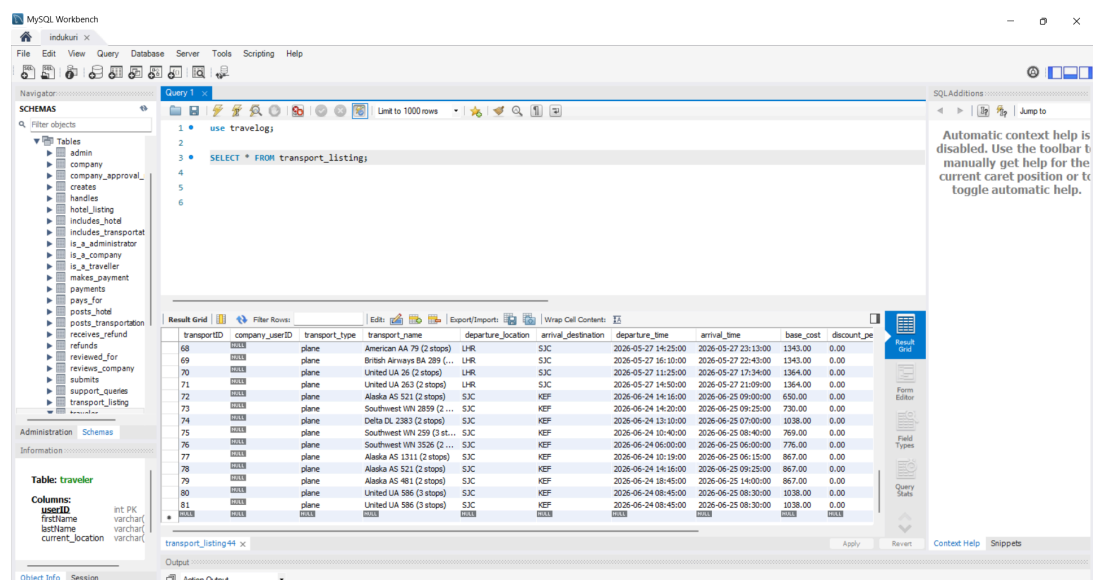


Figure 9.4: Browser Showing Results

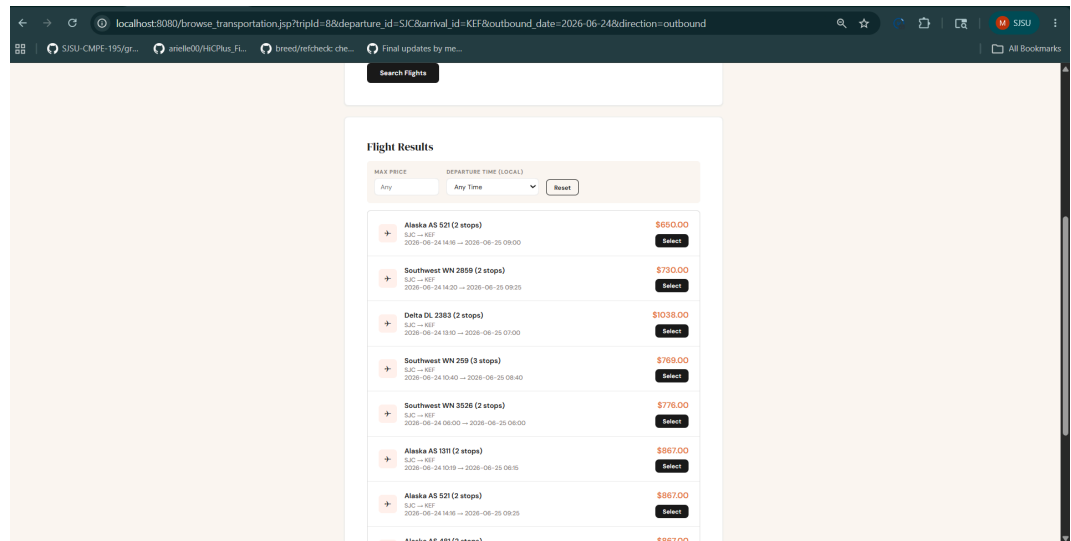
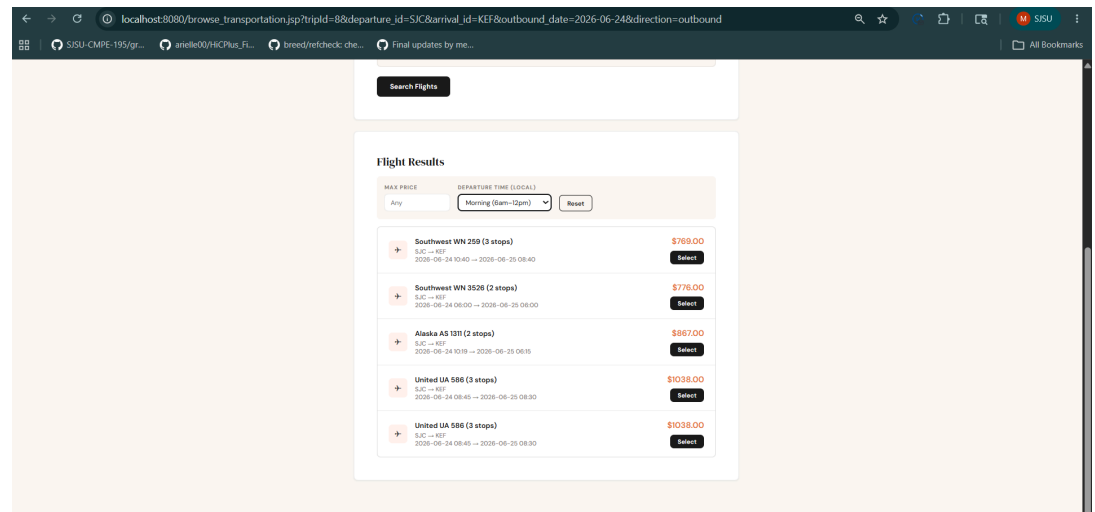


Figure 9.4: Filtering (Client Side using JavaScript for efficiency)



#### f. Select Transportation

Files: select\_transportation.jsp

Tables Affected: Trip\_Transportation, Includes\_Transportation

SQL Operations:

- DELETE (overwriting the existing selections in same direction)
- INSERT into Trip\_Transportation
- INSERT into Includes\_Transportation

Implementation Process:

- Lookup the base cost from the Transport Listing
- Delete planned rows that were already selected for that tripID and direction
- Compute next sequence\_num
- Insert into both Trip\_Transportation + Includes\_Transportation

Figures:

Figure 10.1: Browser Selecting on OutBound Flight

The screenshot shows a web browser window with a URL bar indicating a local development environment. The page displays a 'Trip Details' form with the following fields:
 

- TRIP NAME: Iceland Trip
- FROM: San Jose
- TO: Reykjavik
- START DATE: 06/24/2026
- END DATE: 07/29/2026

 A 'Save Changes' button is located below the form. Below the form is an 'Outbound Flight' section showing a flight from San Jose to Reykjavik on 06/24/2026, with a price of \$760.00. A 'Browse Outbound Flights' button is at the bottom of this section.

Figure 10.2: Workbench Queries Showing Inserted Rows

The top screenshot shows a MySQL Workbench window with a query editor containing the following SQL code:
 

```
1 use travelog;
2
3 SELECT * FROM trip_transportation WHERE tripID = 1;
4
5
6
```

 The 'Result Grid' shows the following data:
 

| trip_transportation | tripID | transportID | direction | price  | booking_status |
|---------------------|--------|-------------|-----------|--------|----------------|
| 1                   | 1      | 75          | outbound  | 760.00 | planned        |

 The bottom screenshot shows the same MySQL Workbench window with a different query:
 

```
1 use travelog;
2
3 SELECT * FROM includes_transportation WHERE tripID = 8;
4
5
6
```

 The 'Result Grid' shows the following data:
 

| includes_transportation | tripID | transportID |
|-------------------------|--------|-------------|
| 1                       | 8      | 75          |

### g. Browse and Filter Hotel Options

Files: browse\_hotels.jsp

Tables Affected: Hotel\_Listing

SQL Operations:

- SELECT (cache checking, which I chose 30 days for)
- INSERT (cache API results)
- DELETE (delete old/stale/outdated rows)

Implementation Process:

- Same exact pattern as Browse Transport

Figures:

Figure 11.1: Browser Hotel Search Form Filled Out

The screenshot shows a web browser window with the URL `localhost:8080/browse_hotels.jsp?tripId=8`. The page features a search form for hotels. The form has a title 'Search Hotels & Lodging' and a subtitle 'Fill in the information below'. It includes a 'Location' field with a dropdown menu set to 'Repeat', a 'Number of Adult Guests' field with a dropdown menu set to '2', a 'Check In Date' field with a calendar icon set to '06/24/2008', and a 'Check Out Date' field with a calendar icon set to '07/29/2008'. A 'Search Hotels' button is located at the bottom of the form. The browser's address bar shows the URL, and the page title is 'Hotels. Search hotels via Google Hotels.'.

Figure 11.2: Workbench Query Before Search

The screenshot shows the MySQL Workbench interface. The 'Query' window contains the SQL query `SELECT * FROM hotel_listing;`. The 'Result' window displays a table with the following data:

| hotelID | company_userID | hotel_name                                 | location | description                                         | rating | price_per_night | discount_percent |
|---------|----------------|--------------------------------------------|----------|-----------------------------------------------------|--------|-----------------|------------------|
| 318     |                | Disney's Animal Kingdom Lodge              | REVIEWAK | Safari-themed Disney property in a wildlife park... | 4.8    | 0.00            | 0.00             |
| 319     |                | Sandpiper Resort                           | REVIEWAK | Polished lodging offering refined rooms with wat... | 4.5    | 0.00            | 0.00             |
| 320     |                | Wild Disney World Dolphin Resort           | REVIEWAK | Polished waterfront resort on 87 acres, offering... | 4.3    | 0.00            | 0.00             |
| 321     |                | Disney's Art of Animation Resort           | REVIEWAK | Animation-themed resort set in Walt Disney W...     | 4.6    | 0.00            | 0.00             |
| 322     |                | Disney's All-Star Music Resort             | REVIEWAK |                                                     | 4.5    | 0.00            | 0.00             |
| 323     |                | Gaillard Palmis Resort & Convention Center | REVIEWAK | Warm, unfussy quarters on 63 acres with lush l...   | 4.4    | 0.00            | 0.00             |
| 324     |                | Margaritaville Resort Orlando              | REVIEWAK | Sprawling resort offering quirky lodgings, a nat... | 4.3    | 0.00            | 0.00             |
| 325     |                | The Grove Resort & Water Park Orlando      | REVIEWAK | Upsmarket resort offering 3 restaurants, a cafe ... | 4.0    | 0.00            | 0.00             |
| 326     |                | The Island Resort at Fort Walton Beach     | REVIEWAK | Lively modern hotel with 2 pools & 5-story wa...    | 4.0    | 0.00            | 0.00             |
| 327     |                | Disney's Pop Century Resort                | REVIEWAK | Warm rooms in a colorful nostalgia-themed res...    | 4.5    | 0.00            | 0.00             |
| 328     |                | Vynoham Grand Clearwater Beach             | REVIEWAK | Cozy rooms in an upscale property offering gulf...  | 4.5    | 0.00            | 0.00             |
| 329     |                | The Sun Cofee                              | REVIEWAK | Beachfront lodging & spa with upscale rooms & ...   | 4.4    | 0.00            | 0.00             |
| 330     |                | Shades of Green                            | REVIEWAK |                                                     | 4.7    | 0.00            | 0.00             |
| 331     |                | Oona Amelia Island Resort & Spa            | REVIEWAK | Elegant oceanfront resort offering golf courses ... | 4.3    | 0.00            | 0.00             |
| 332     |                |                                            |          |                                                     |        |                 |                  |

Figure 11.3: Workbench Query After Search

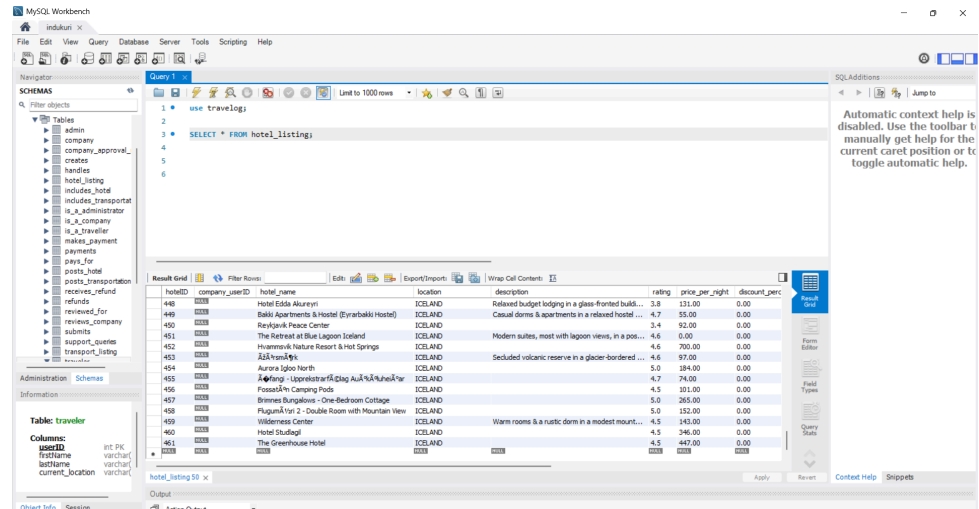
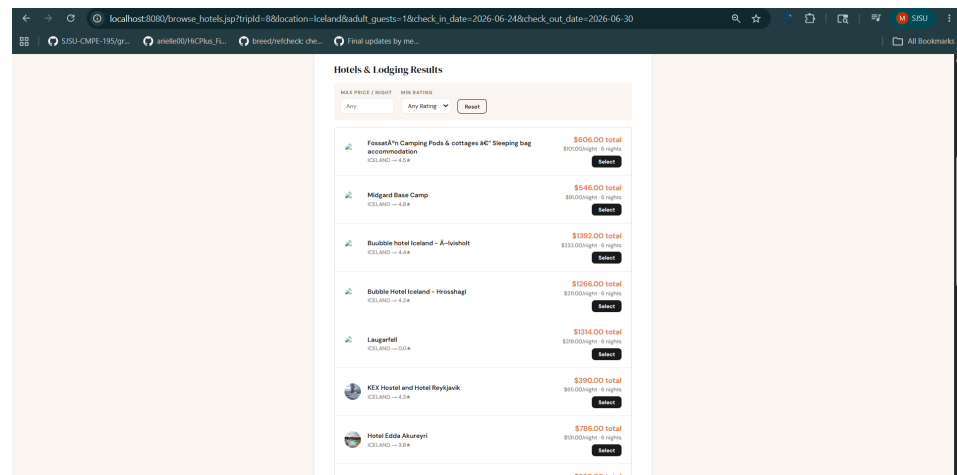


Figure 11.4: Browser Hotel Results



## h. Select Hotel

Files: select\_hotel.jsp

Tables Affected: Trip\_Hotels, Includes\_Hotel

SQL Operations:

- SELECT COUNT (date overlap check)
- INSERT into Trip\_Hotels
- INSERT into Includes\_Hotels

Implementation Process:

- Validate that check-in and checkout don't overlap with an already existing hotel on the trip
- Calculate total cost (nights \* price per night)
- Insert into both Trip\_Hotels + Includes\_Hotel

Figures:

Figure 12.1: Browser Selecting on Hotel

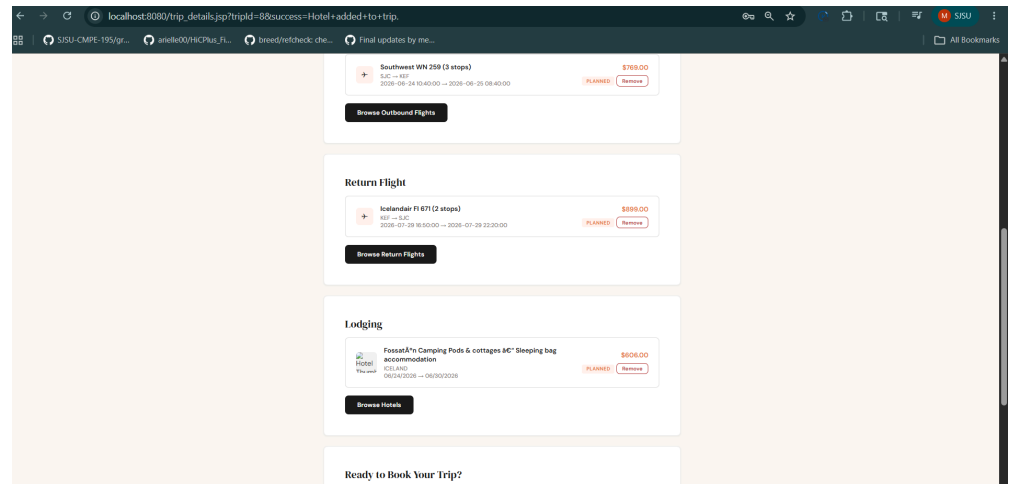
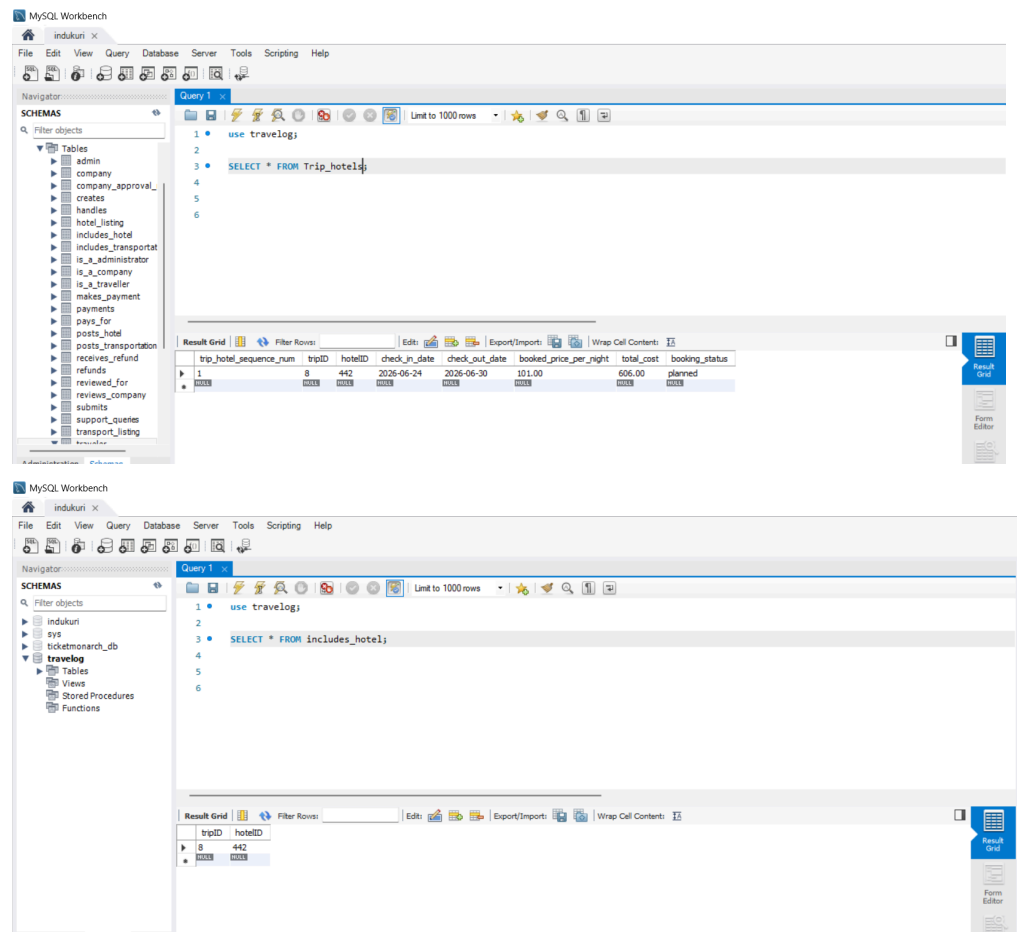


Figure 12.2: Workbench Queries Showing Inserted Rows



## i. Review and Confirm Trip Booking

Files: confirm\_trip\_details.jsp, payment\_status\_process.jsp

Tables Affected: Trip, Trip\_Hotels, Trip\_Transportation, Payments, Makes\_Payment, Pays\_FOR

SQL Operations:

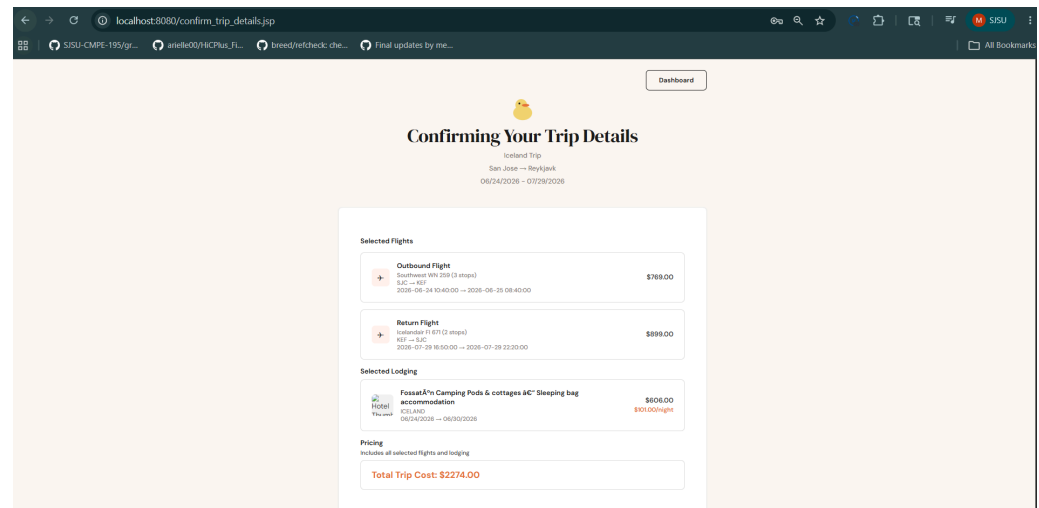
- SELECT (sum costs)
- INSERT into Payments
- INSERT into Makes\_Payments + Pays\_For (relation tables)
- UPDATE Trip status to booked
- INSERT into Trip\_Hotels + Trip\_Transportation booking status changed

Implementation Process:

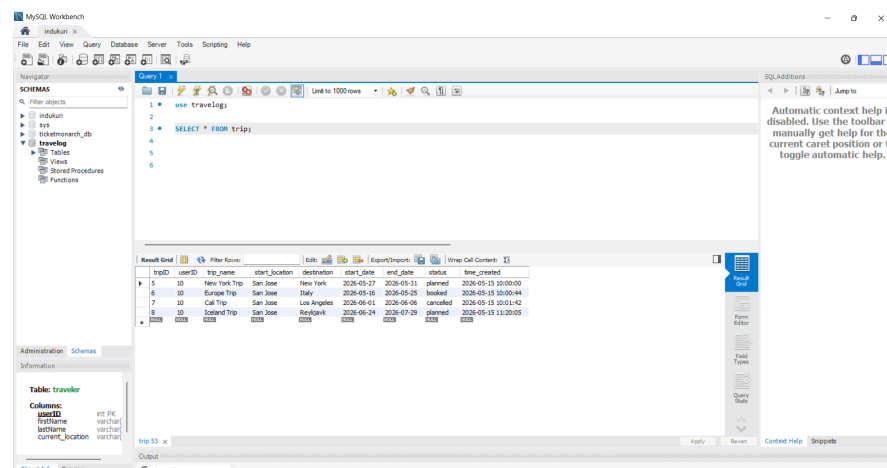
- Calculate the grandTotal from Trip\_Hotels and Trip\_Transportation
- Collect payment method + (non-stored) card details
- INSERT into Payments, Makes\_Payment, and Pays\_For
- Change all related booking status fields to booked
- Redirect to receipt.jsp

Figures:

**Figure 13.1: Browser Showing Confirm Trip Details Page**



**Figure 13.2: Workbench Before Trip & Payments Table**





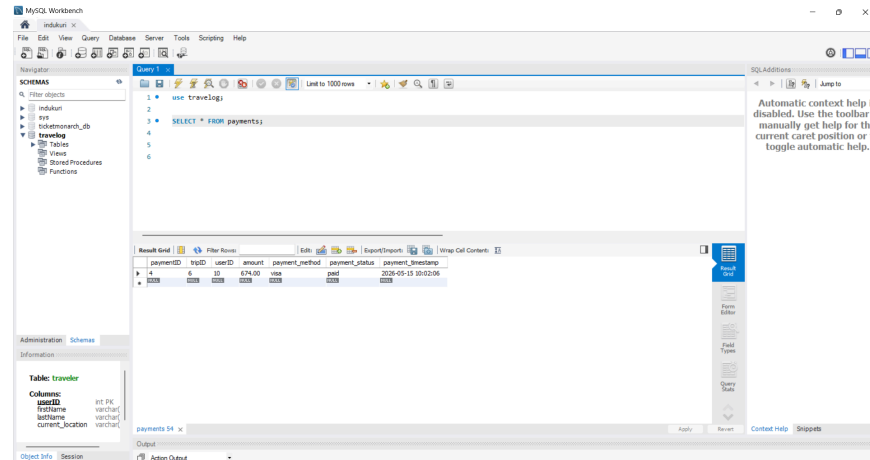


Figure 13.3: Browser with Payment Details

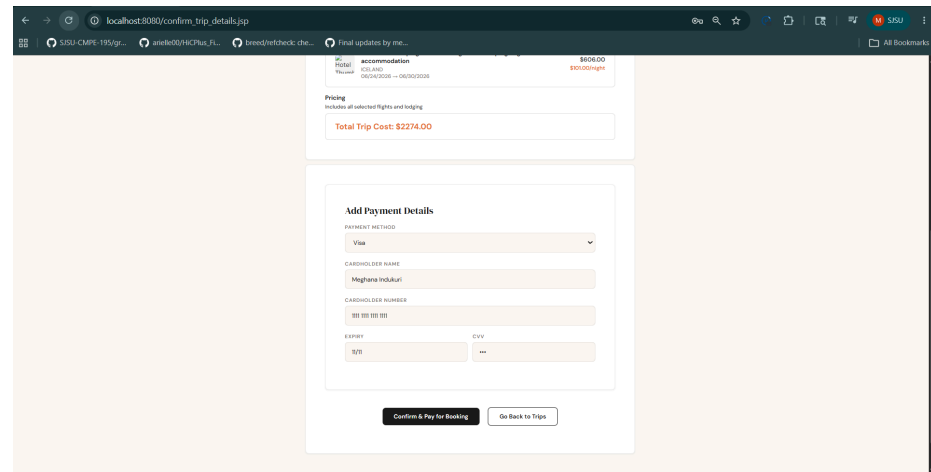
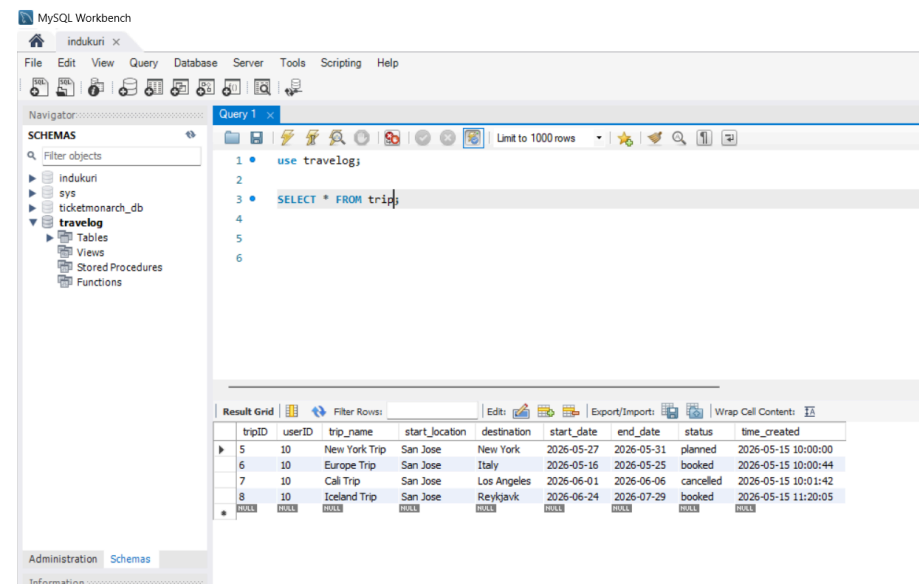


Figure 13.4: Work Bench Tables After Submitting Payment



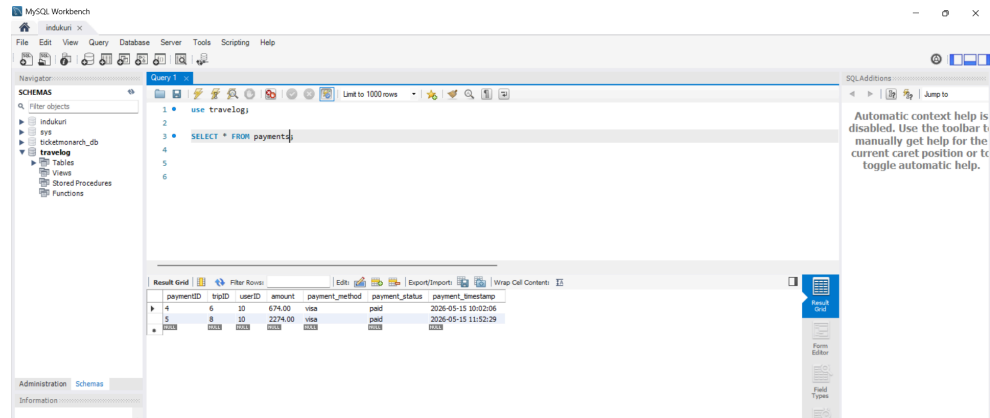
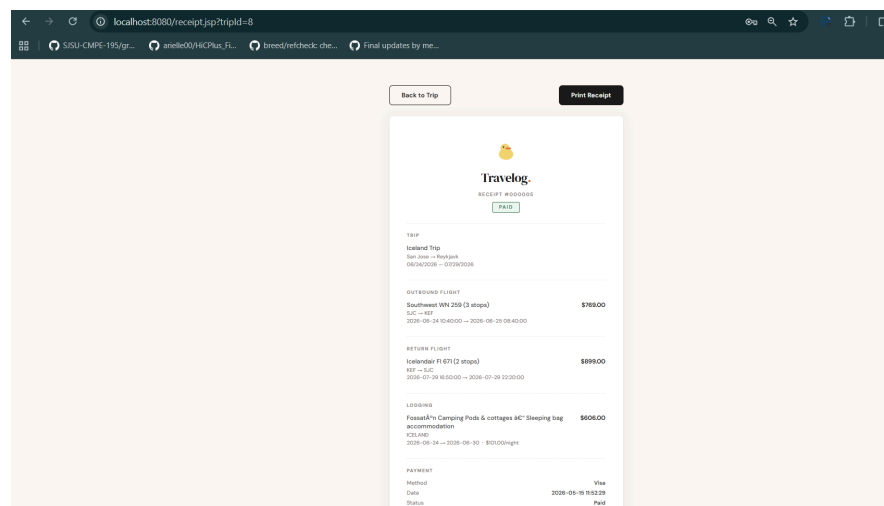


Figure 13.5: Receipt Page



#### j. Cancel Trip and Receive Refund

Files: `trip_details.jsp`, `cancel_trip_process.jsp`, `remove_hotel.jsp`, `remove_transportation.jsp`

Tables Affected: Trip, Trip\_Hotels, Trip\_Transportation, Payments, Refunds, Receives\_Refund

SQL Operations:

- SELECT (verify password and lookup the payment)
- UPDATE Trip, and other status fields of other tables to cancelled and refunded
- INSERT into Refunds + Receives\_Refund

Implementation Process:

- User chooses the full-trip cancel or the per-item cancel

- They confirm using password
- Update statuses, insert refunds row that is equal to the amount they refunding
- Row is also inserted into Receives\_Refund

Figures:

Figure 14.1: Browser Showing Confirm Trip Details Page

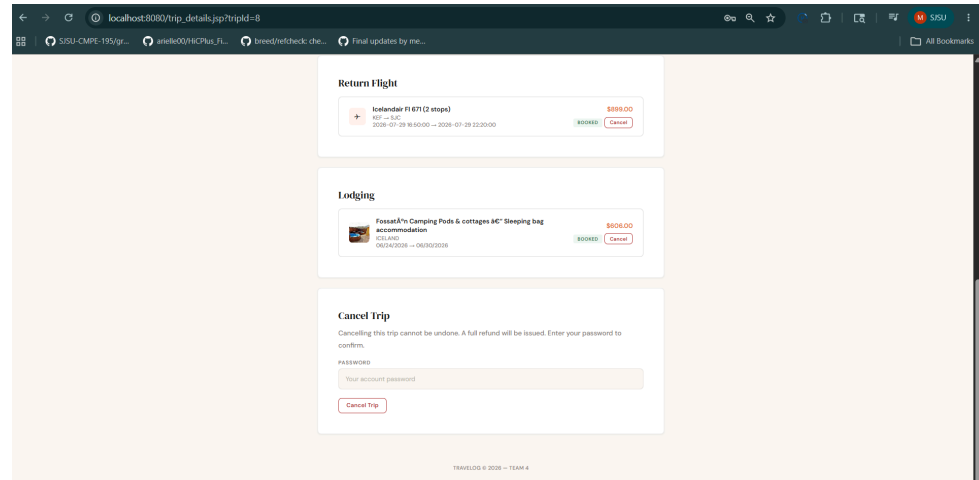
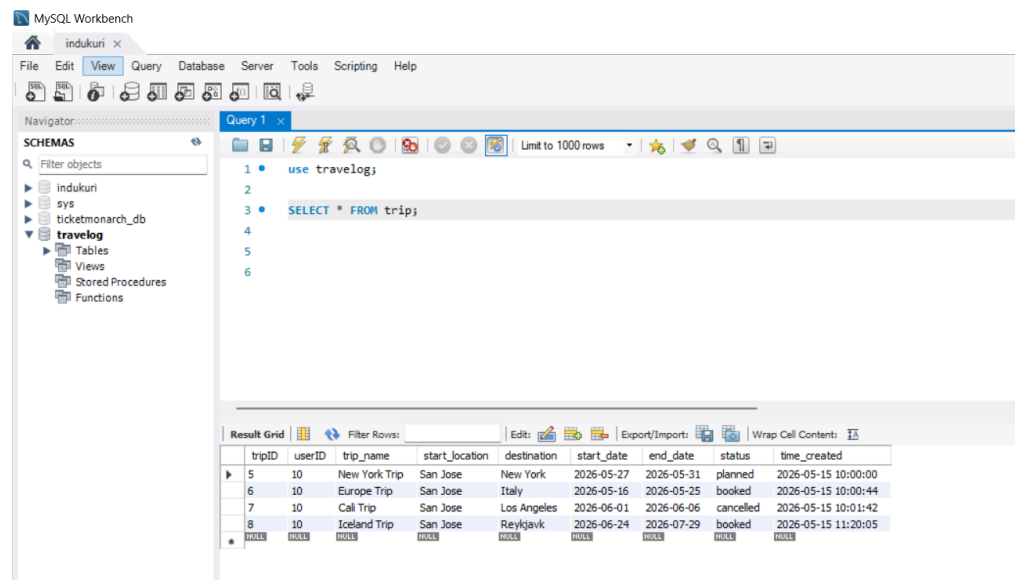


Figure 14.2: Workbench Before (Trip, Refunds Tables)



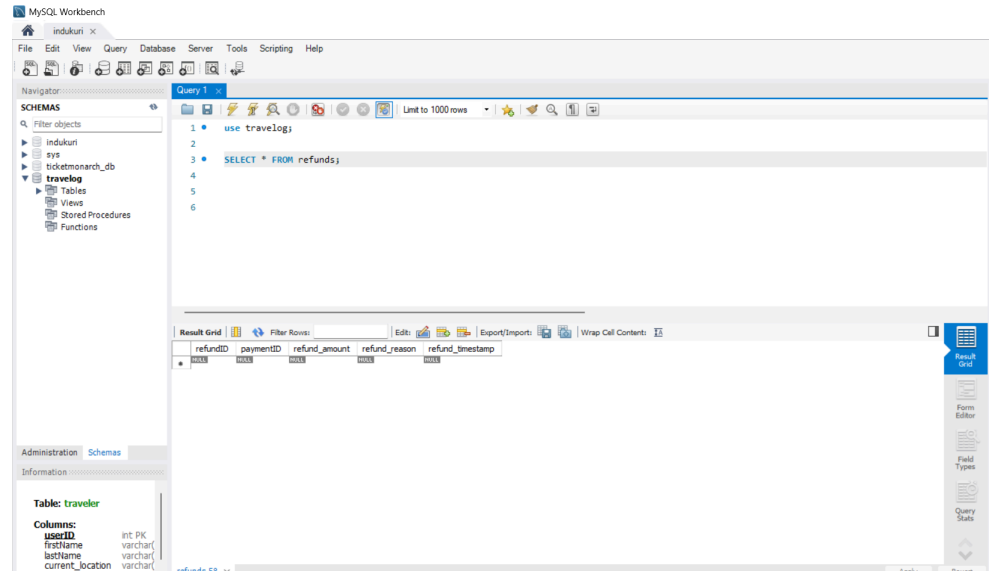


Figure 14.3: Browser with Receipt After Successful Confirmation

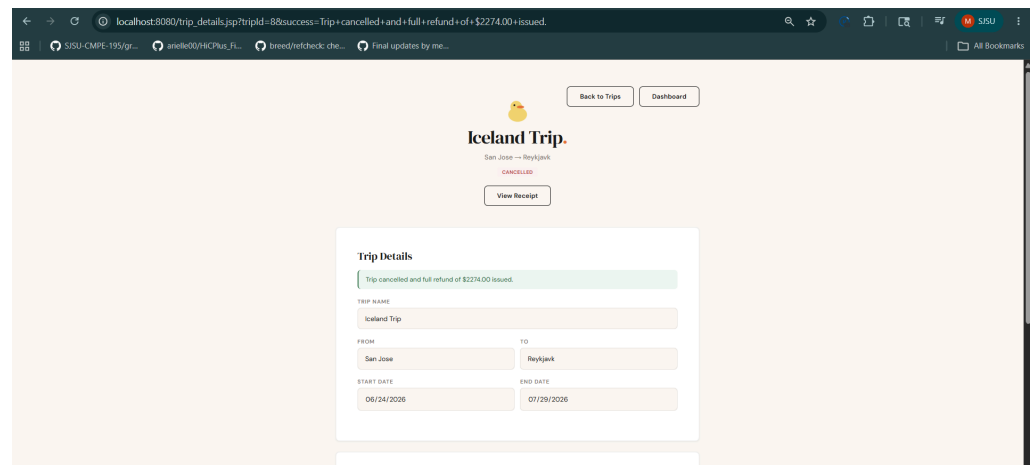
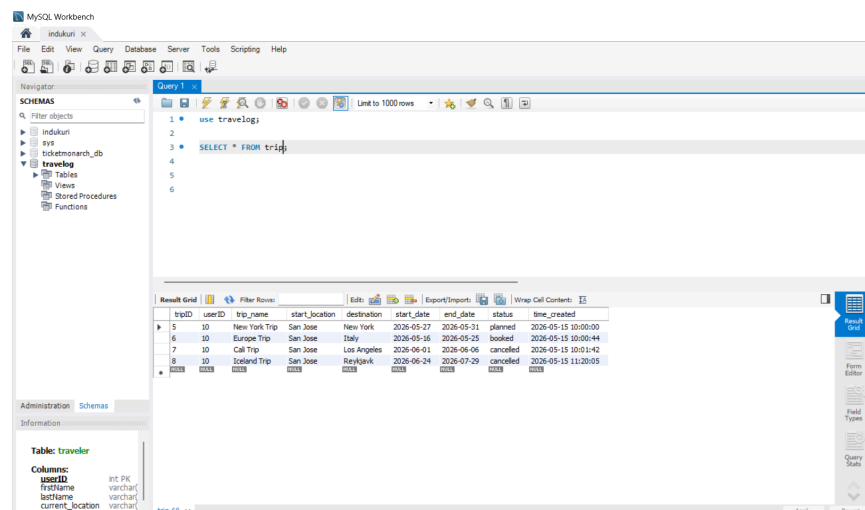
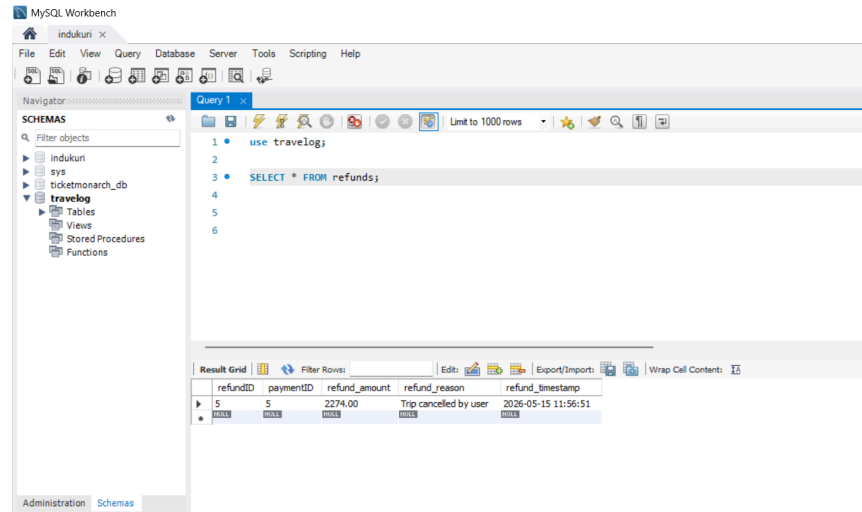


Figure 14.4: Work Bench After (Trip and Refunds Table)





### 3. Company View

#### a. Register Account, Account Sign in, Account Logout, Delete Account

Same implementation as the Traveler functionalities (above). Here the companies are inserted into Company + Is\_A\_Company tables and approval status is set to pending because they require admin review before they can post the listings.

Figures:

Figure 15.1: Browser Create Account Company Form Filled

The screenshot shows a web browser at the URL `localhost:8080/register.jsp`. The page has a light orange background and a white central form titled 'Get started.' with the subtitle 'Create your Travelog account'. The form contains the following fields and options:

- USERNAME:
- EMAIL:
- PASSWORD:
- ACCOUNT TYPE:
- COMPANY NAME:
- BUSINESS TYPE:
- PHONE (OPTIONAL):

At the bottom of the form is a 'Create Account' button. Below the button, there is a link: 'Already have an account? Sign In'.

Figure 15.2: Tables Before (Company & Users & Is\_A\_Company Table)

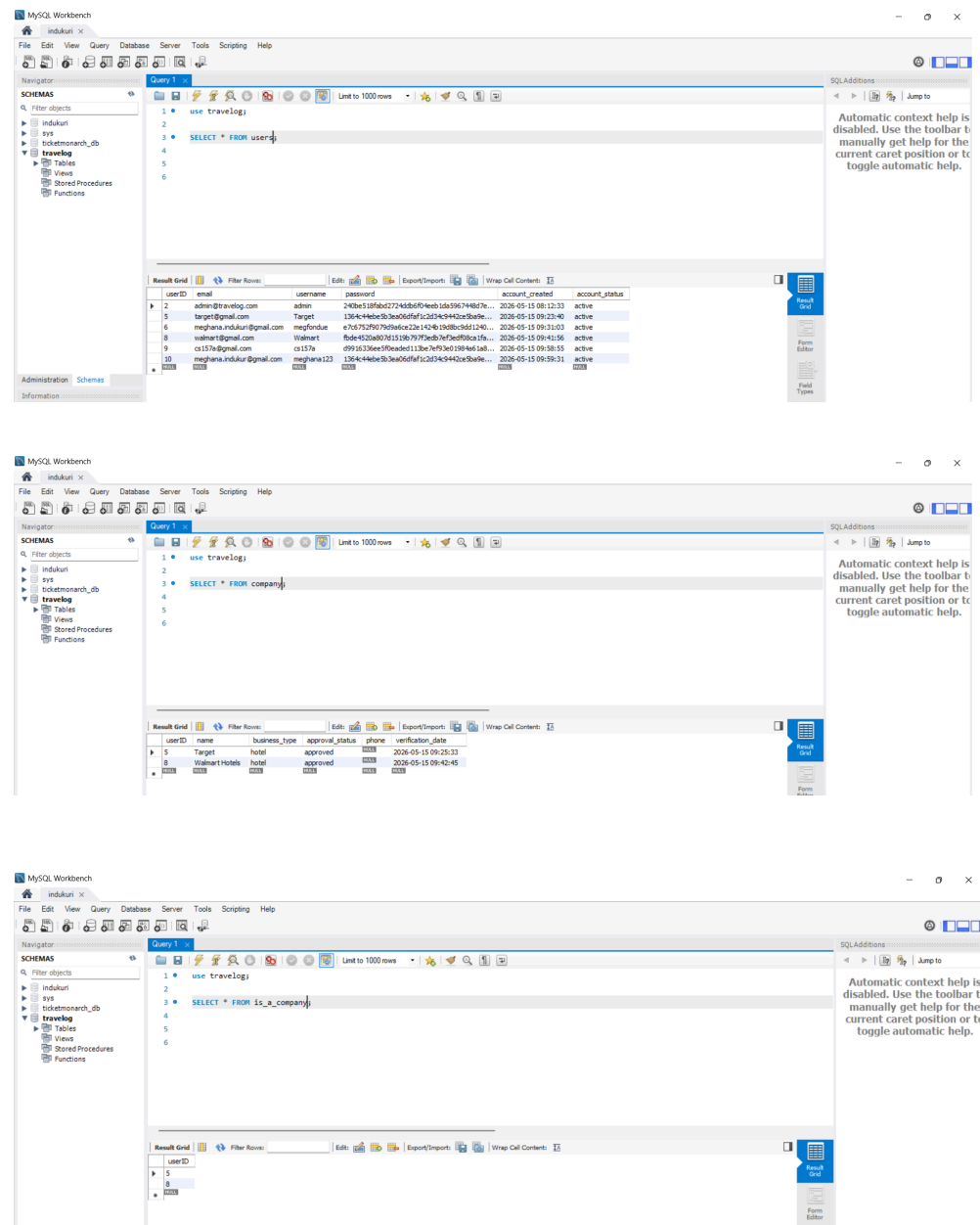


Figure 15.3: Browser Create Account Completed

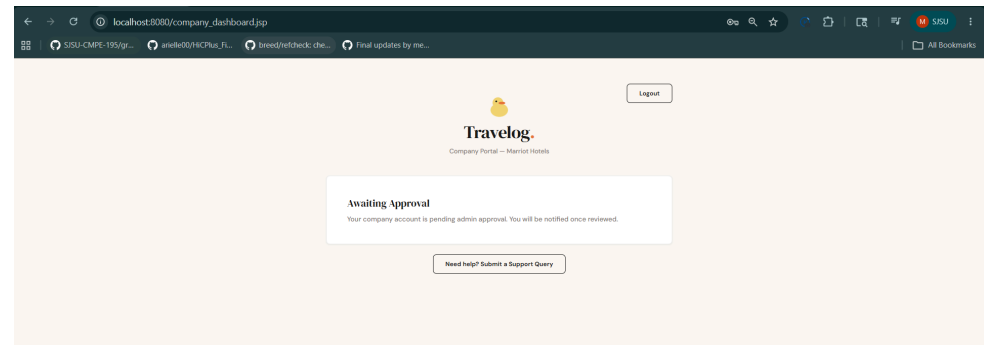
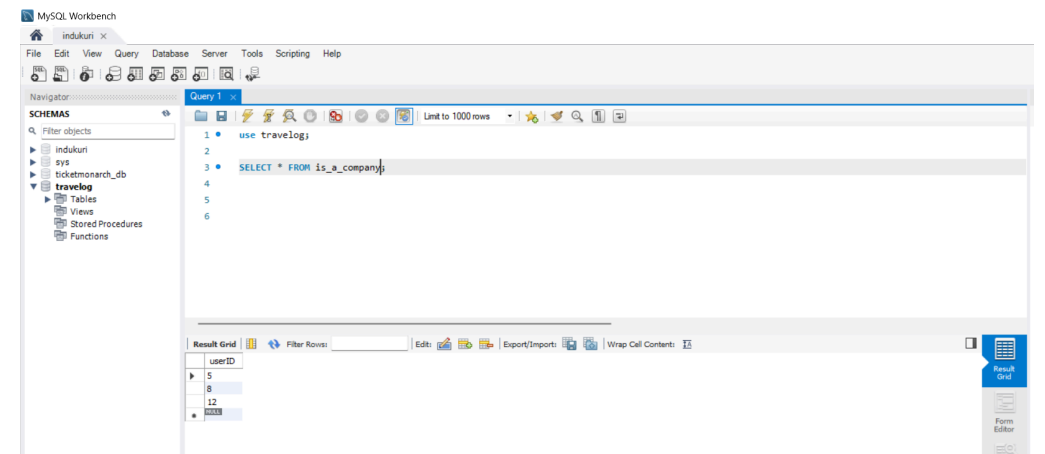
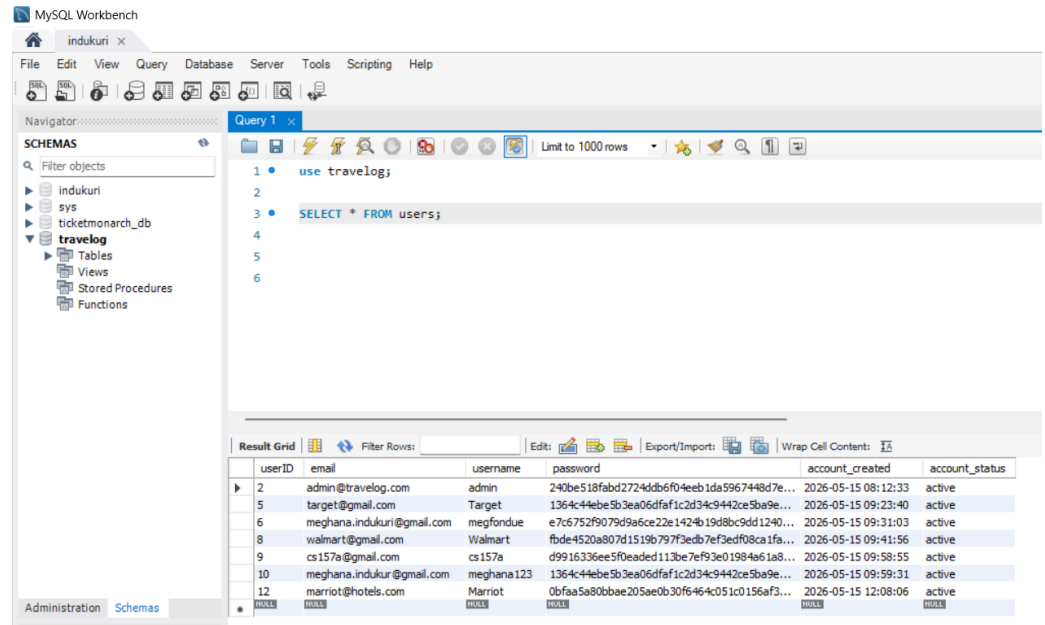
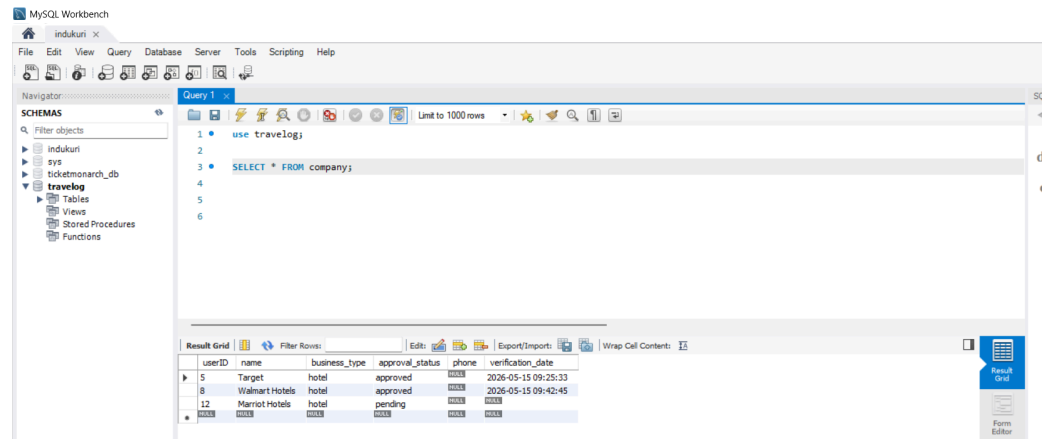


Figure 15.4: Workbench After



## b. Post Offer

Files: company\_dashboard.jsp, post\_hotel\_process.jsp,  
post\_transportation\_process.jsp

Tables Affected: Hotel\_Listing or Transport\_Listing, Posts\_Hotel or  
Posts\_Transportation

SQL Operations:

- INSERT into listing table (with discount percent and availability)
- INSERT into relation table as well

Implementation Process:

- Validate the required fields and the range of the discount
- Insert into Hotel\_Listing or Transport\_Listing
- Insert into Hotel\_Listing or Transport\_Listing
- Insert into corresponding Posts\_ relation table

Figures:

Figure 16.1: Browser Showing Post Information Filled In

The screenshot shows a web browser window with the URL 'localhost:8080/company\_dashboard.jsp'. The page has a 'Travelog' logo and a 'Login' button. The main content is a form titled 'Post a Hotel Listing'. The form fields are as follows:

| Field                 | Value             |
|-----------------------|-------------------|
| HOTEL NAME            | Marriott Downtown |
| LOCATION (CITY)       | San Jose, CA      |
| DESCRIPTION           | Cool Hotel        |
| PRICE PER NIGHT (USD) | 15099             |
| RATING (0-5)          | 4.9               |
| DISCOUNT % (0-100)    | 10                |
| AVAILABILITY          | Available         |

A 'Post Hotel' button is located at the bottom of the form.

Figure 16.2: Workbench Showing New Rows

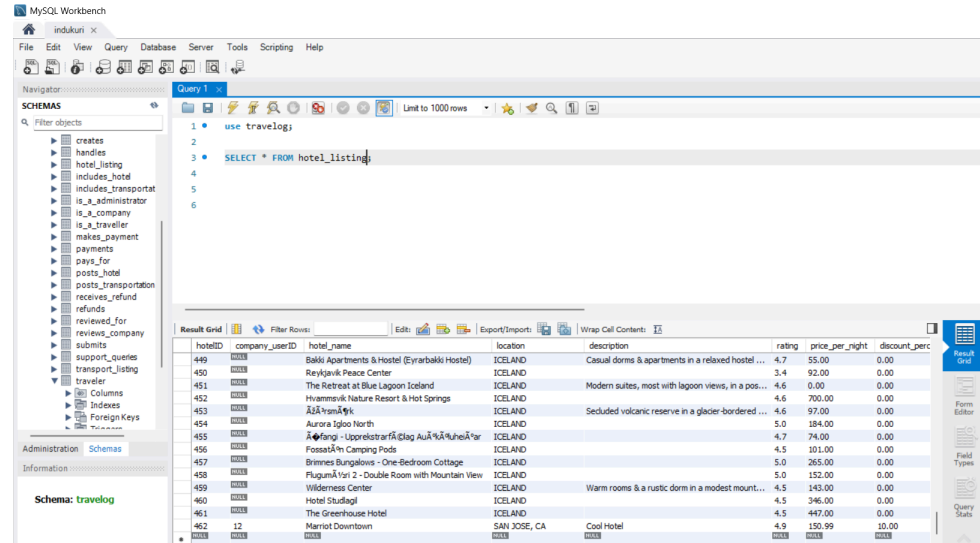
The screenshot shows a database workbench interface. The 'Schemas' pane on the left shows the 'travelog' database selected. The 'Query' pane shows the following SQL query:

```
1 use travelog;
2
3 SELECT * FROM posts_hotel;
4
5
6
```

The 'Result Grid' pane shows the following data:

| company_userID | hotelID |
|----------------|---------|
| 5              | 192     |
| 8              | 293     |
| 12             | 462     |





### c. Delete Post

Files: company\_dashboard.jsp, delete\_listing\_process.jsp

Tables Affected: Hotel\_Listing or Transport\_Listing

SQL Operations:

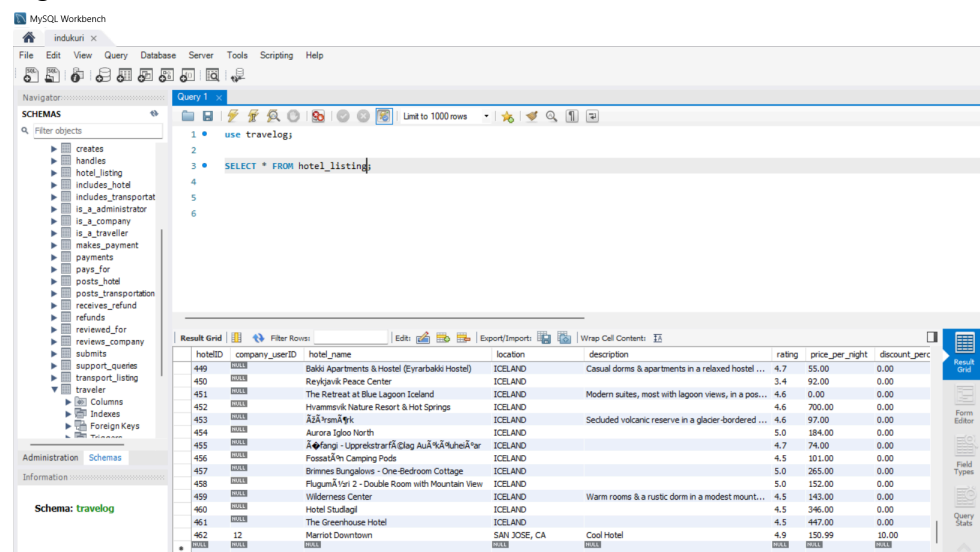
- SELECT (verify password)
- UPDATE listing status to archived

Implementation Process:

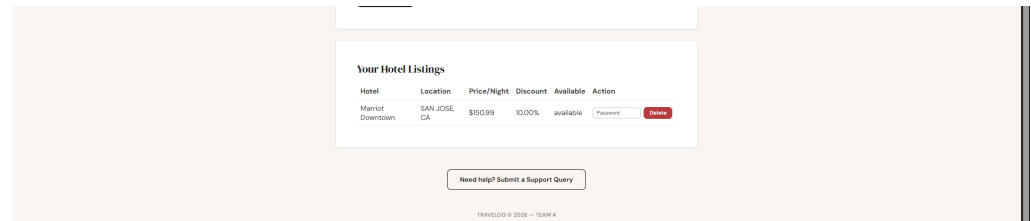
- Confirm using password of archival/delete process
- Update the listing status to archived so existing bookings saved

Figures:

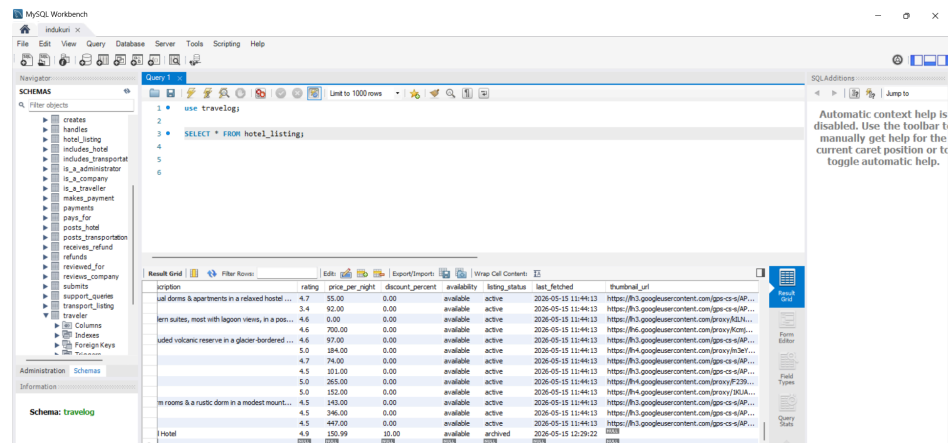
Figure 17.1: Workbench Before



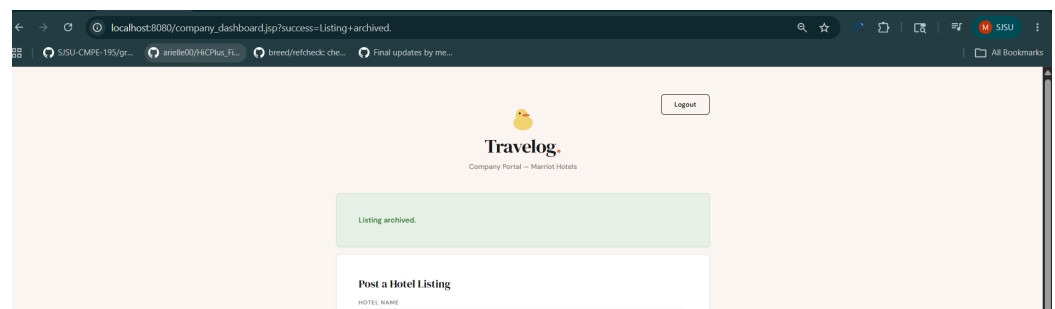
**Figure 17.2: Browser Showing Delete Company Listings UI**



**Figure 17.3: Workbench After Deletion/Archivation**



**Figure 17.4: Browser Showing Archived Banner**



#### 4. Administrator View

##### a. Confirm or Reject Company Accounts

Files: admin\_dashboard.jsp, review\_company\_process.jsp

Tables Affected: Company, Company\_Approval\_Reviews, Reviews\_Company, Reviewed\_For

SQL Operations:

- SELECT (verify admin)

- UPDATE Company.approval\_status and the verification date
- INSERT into Company\_Approval\_Reviews
- INSERT into Reviews\_Company + Reviewed\_For (relation tables)

Implementation Process:

- Admin sees the list of pending companies
- Admin chooses to approve or reject
- Update Company and insert review and relation rows

Figures:

Figure 18.1: Tables Before (Company and Company\_Approval\_Reviews)

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists the 'travelog' database. The 'Query' editor shows the following SQL query:

```
1 use travelog;
2
3 SELECT * FROM company;
4
5
6
```

The 'Result Grid' displays the following data:

| userID | name           | business_type | approval_status | phone | verification_date   |
|--------|----------------|---------------|-----------------|-------|---------------------|
| 5      | Target         | hotel         | approved        | 0000  | 2026-05-15 09:25:33 |
| 8      | Walmart Hotels | hotel         | approved        | 0000  | 2026-05-15 09:42:45 |
| 12     | Mammoth Hotels | hotel         | pending         | 0000  | 0000                |

The screenshot shows the MySQL Workbench interface. The 'Query' editor shows the following SQL query:

```
1 use travelog;
2
3 SELECT * FROM company_approval_reviews;
4
5
6
```

The 'Result Grid' displays the following data:

| reviewID | company_userID | admin_userID | decision | review_notes | review_timestamp    |
|----------|----------------|--------------|----------|--------------|---------------------|
| 1        | 5              | 2            | approved | 0000         | 2026-05-15 09:25:33 |
| 2        | 8              | 2            | approved | 0000         | 2026-05-15 09:42:45 |

Figure 18.2: Browser Showing Pending Company Approvals

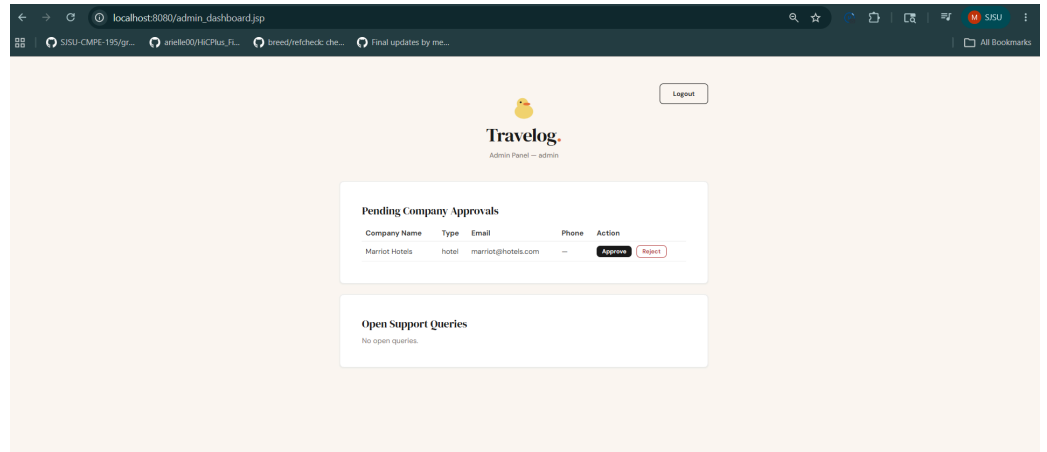


Figure 18.3: Workbench After Confirmation

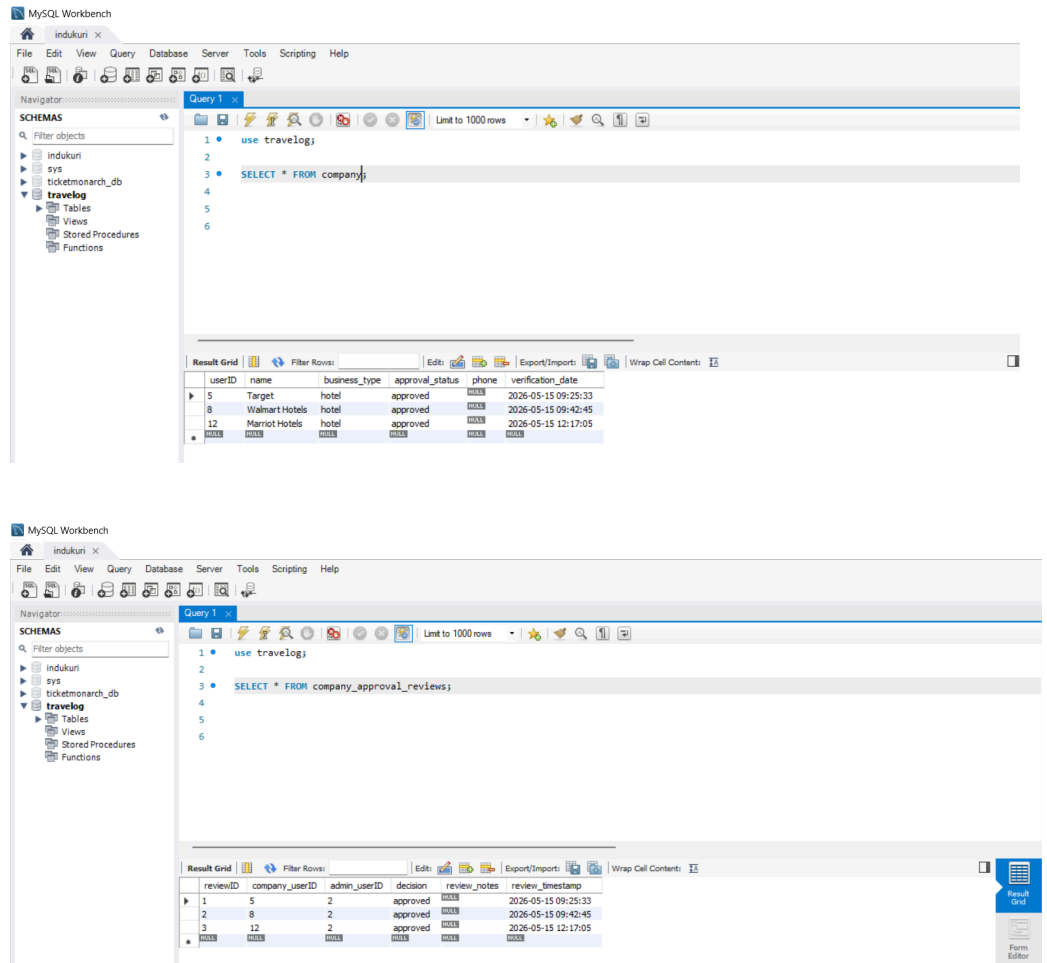
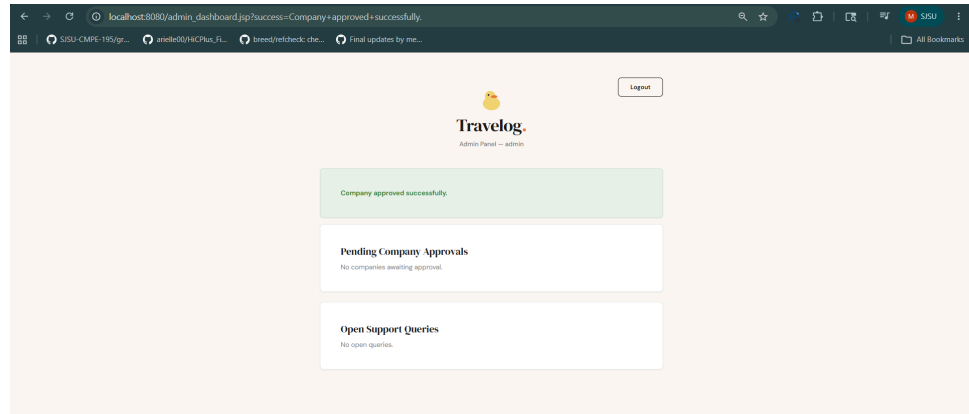


Figure 18.4: Browser Updated After Confirmation



## b. Handle User Queries

Files: admin\_dashboard.jsp, handle\_query.jsp, handle\_query\_process.jsp

Tables Affected: Support\_Queries, Handles

SQL Operations:

- SELECT (verify admin and get the queries)
- UPDATE Support\_Queries status and add resolved timestamp
- INSERT into Handles

Implementation Process:

- Admin selects query from inbox
- Admin chooses new status
- Update the support queries and insert into handles

Figures:

Figure 19.1: Query List and Query Page in Browser

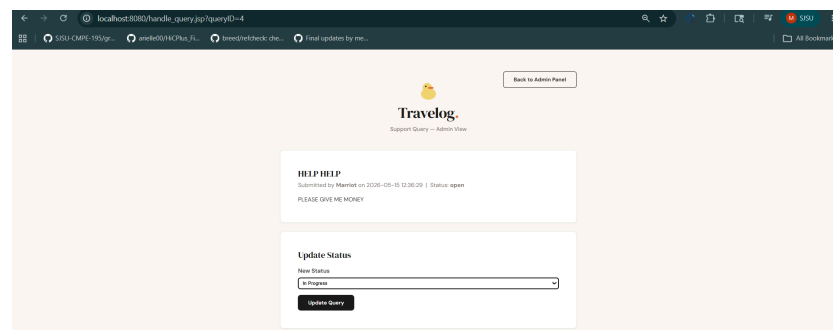
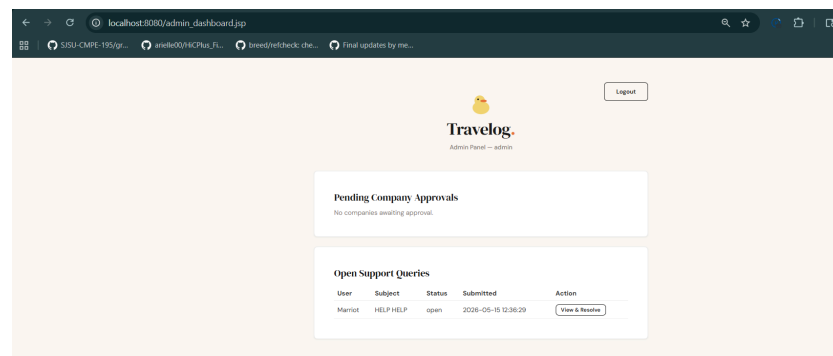
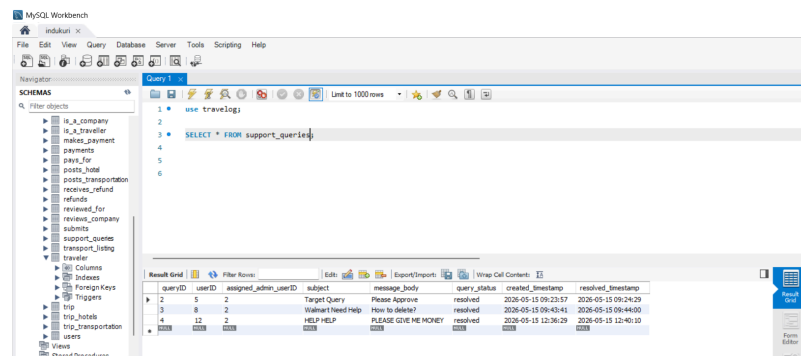


Figure 19.1: Workbench After Resolving Query



## 5. Submit Support Query (used by traveler and company)

Files: submit\_query.jsp, submit\_query\_process.jsp

Tables Affected: Support\_Queries, Submits

SQL Operations:

- INSERT into Support\_Queries with status open
- INSERT into Submits (relation table)

Implementation Process:

- User fills subject and message
- Insert into Support\_Queries and Submits

Figures:

Figure 20.1: Browser Submit Query Form Filled

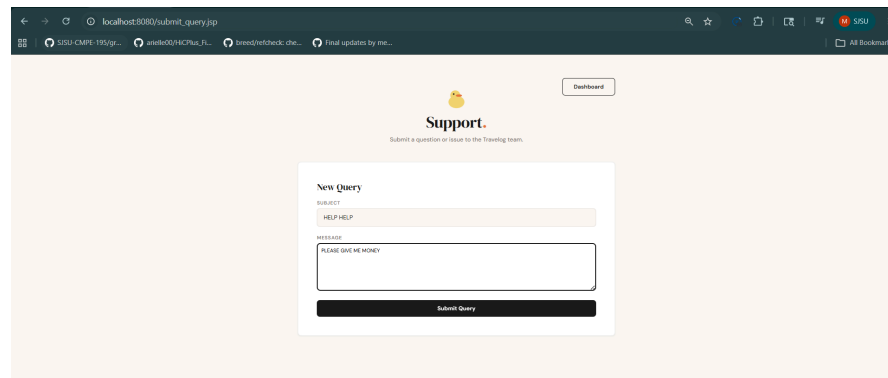
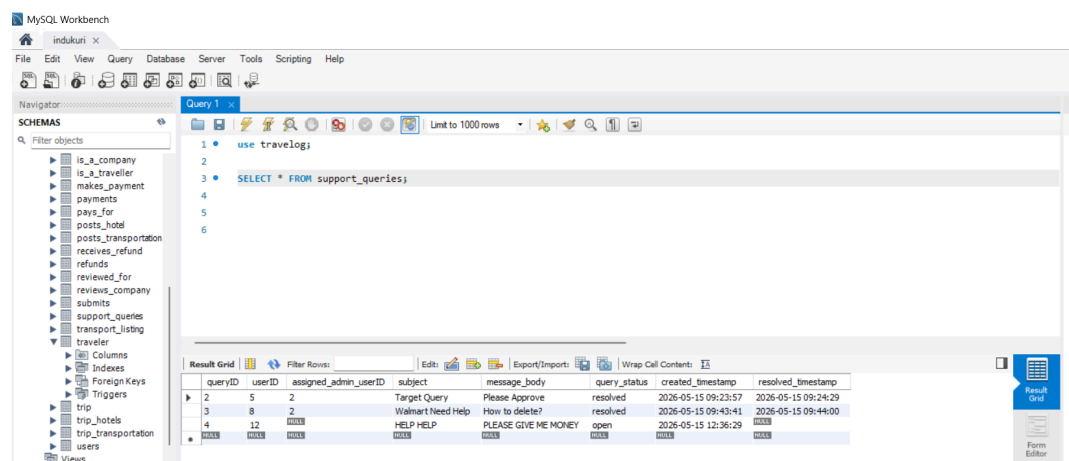


Figure 20.1: Updated Rows in Workbench



## Lessons Learned

### **Meghana Indukuri:**

Through this project, I was able to extend my knowledge of true application development, specifically three-tiered architectures. I was able to utilize my existing knowledge of Java and combine it with relational databases, such as MySQL, to create a working prototype of a travel application. Furthermore, I have become more proficient in frontend development and all it entails. I was able to use JavaScript, HTML, and CSS to create pages that are easy to use while also improving the efficiency of backend calls. I also had the opportunity to explore free APIs, and I found SerpAPI, which allowed me to use Google Flights to get real flight data from one destination to another. I learned how to make API requests, parse JSON responses, and insert the data into our database, which is a valuable real-world skill. It was very fun to learn that pipeline and experiment with different approaches. I also had to think about scalability. Right now, our application only stores 20 results from the API calls, but a question that came up for me was how we would handle a million or even more. This project really made me think about such intricacies, which could not have been possible by only building theoretical knowledge. It was also fun to write queries and see how much the logic sometimes differs from regular programming languages. It is a skill that needs to be developed, and thanks to this project, I was able to improve upon it. Overall, there were many things I learned from this project that I believe will extend to my real-world experiences and roles.

Features I contributed:

- Traveler Registration Flow
- Flight Booking & Filtering Options
- Deleting Accounts
- Refunds

### **Isaiah Villanueva**

Working on this project gave me hands-on experience with Java JSP web application, MySQL relational database and gave me an opportunity to work with a wonderful team. I learned how to write PreparedStatements to insert and query data. I learned how to create an entity diagram but on a large scale. Definitely gave me a different perspective on how database architecture works and how it's very important when planning a project out. Worked on the Company/Support query side of the application where I designed and implemented Company registration flow in order for hotel listing to be put up on the website. This project really gave me a good feel of how it would be to work at a big company where you get put into a team and you need to collaborate with your team in order to succeed. Learning how to debug and set up a database locally and connect to it from my project folder. Overall, a very good project gave me good foundational skills after graduation and maybe at Google someday.

Some Website features I contributed:

- Company registration flow

- Company dashboard with hotel and transportation posting forms
- Company login and session setup

### **Carolyna Sias**

This project was my introduction to full-stack development using a three-tier architecture. I learned to create backend processes using Java/JSP Servlets, using MySQL to perform operations (query/add/update/delete) with user data, and a bit of frontend development using HTML/CSS. One of the initial challenges was setting up each directory to have the app on localhost, but after figuring it out, other challenges were fun to tackle. I hadn't worked with APIs much previously, so integrating Google Hotel data into our app through SerpAPI using JSONs was a highlight in my learning process. I learned about caching and saving searched information into our database for quicker app responses. I liked working on the debugging process and integrating small details that made the app more professional (ex. trip start must be before trip end, adding hotel thumbnail previews, etc.) I enjoyed collaborating with the team, and picked up a few skills from them—many involving authentication, security, and general app-design choices. The most important part was how everything connected from the customer-facing UI to the backend processes into various database operations.

I look forward to using my newfound knowledge and developed skills in future projects!

Some features I worked on:

Creating a Trip

Browsing & Adding Hotel(s) to trip

Viewing Trips (View all trips for a user)

Trip Details (View & confirm details for current trip before proceeding to book)

Adding Payment